

OWASP – Penetration Test Report

Project: OWASP Juice Shop Assessment

Version: 1.0 Date: 01/01/2026 Consultant: Yosef M. El-Manawy

Table of Contents

- **1. Executive Summary**
 - **1.1 Introduction**
 - **1.2 Scope of Work**
 - **1.3 Risk Summary**
 - **1.4 Strategic Recommendations**
- **2. Vulnerability Write-Ups**
 - **2.1 A01:2025 - Broken Access Control**
 1. Privilege escalation by adding the ("role":"admin") during registration process (Critical)
 2. Administration panel path exposed (High)
 3. Sensitive data exposure /ftp/ endpoint (High)
 4. Exposed valid testing credentials in the client side (High)
 5. Backup files exposed and access is possible through null byte injection (High)
 6. Business logic flaw by inserting negative value of the quantity parameter (High)
 7. IDOR leads to unauthorized access to other users baskets (High)
 8. IDOR leads to adding items to other users baskets (High)
 9. IDOR leads to deleting other users items from the basket (High)
 10. Business logic flaw leads to submit "none" as a payment method in the membership functionality (High)
 11. Sensitive information leakage (metrics) (Medium)
 12. Product tampering in the products API (Medium)
 13. Business logic flaw leads to submit negative or exceed the positive value of rating in the feedback functionality (Medium)
 14. Submitting complains on behalf of other users in the customer support form (complain) (Medium)
 15. IDOR leads to unauthorized access to the order confirmation pdf of another user by replacing the UUID (Medium)

16. IDOR leads to unauthorized access to tracking order of other users through replacing the UUID (Medium)
 17. Submitting reviews on other users behalf in the review functionality (Medium)
 18. Outdated URL allow list redirect in the GitHub redirect endpoint (Medium)
 19. Allow list redirect bypass in the GitHub redirect endpoint (Medium)
 20. Submitting feedbacks on behalf of other users through JSON data parameters (Low)
 21. Business logic flaw leads to add more balance to the digital wallet than intended (Low)
 22. CSRF in the username change request in the profile page (Low)
 23. It's possible for one user to access another user's data if the 2 users have a similar email addresses in the request data export functionality (Low)
 24. IDOR leads to unauthorized access to unreleased items (Low)
 25. Registering an account with an empty passwordRepeat field (None)
- **2.2 A04:2025 - Cryptographic Failures**
 26. Clear text communication over HTTP (High)
 27. Weak hashing algorithm (md5) for passwords (Medium)
 - **2.3 A05:2025 - Injection**
 28. SQLi via email field of the login page (Critical)
 29. SQLi in the search API (High)
 30. XXE via file upload in the complain form (High)
 31. Stored XSS in the email field of registration form: 6.9 (Medium)
 32. DOM XSS in the search functionality (Medium)
 33. Reflected XSS in the URL parameter in the track order page (Medium)
 34. HTTP-Header stored XSS in the "/saveLoginIp" endpoint through the "True-Client-IP" header (Medium)
 35. Stored XSS in the username field in the profile page (Medium)
 - **2.4 A06:2025 - Insecure Design**
 36. Uploading any file type via the complain form (Low)
 37. Uploading any file type via photo wall form (Low)
 38. CAPTCHA bypass in the feedback functionality (Low)
 - **2.5 A07:2025 - Authentication Failures**
 39. Brute force users accounts passwords in the login form (High)
 40. Access of a deleted account (High)
 41. Enumerating valid emails through registration form due to lack of rate limiting (Low)
 - **2.6 A10:2025 - Mishandling of Exceptional Conditions**

- 42. Verbose error message exposing database type and SQL query in the login endpoint (Medium)
- 43. Verbose error message exposing SQL database type and the query statement in add to basket functionality (Medium)
- 44. Verbose error message leaking the database type through feedback functionality (Medium)
- 45. Verbose error message exposing valid email in register functionality (Low)*

- **3. Appendices**

- **3.1 Assessment Scope**
 - **3.2 Assessment Artefacts**
-

1. Executive Summary

1.1 Introduction

A **white-box security test** was conducted from 28/12/2025 to 1/1/2026 to assess the security posture of the **OWASP Juice-Shop web application**. The test took 5 full working-days (40 hours) to cover as much as possible of the functionalities of both server-side and client-side.

1.2 Scope of Work

- **Target URL:** <http://juice-shop.com:3000>
- **IP Addresses:** [127.0.0.1]

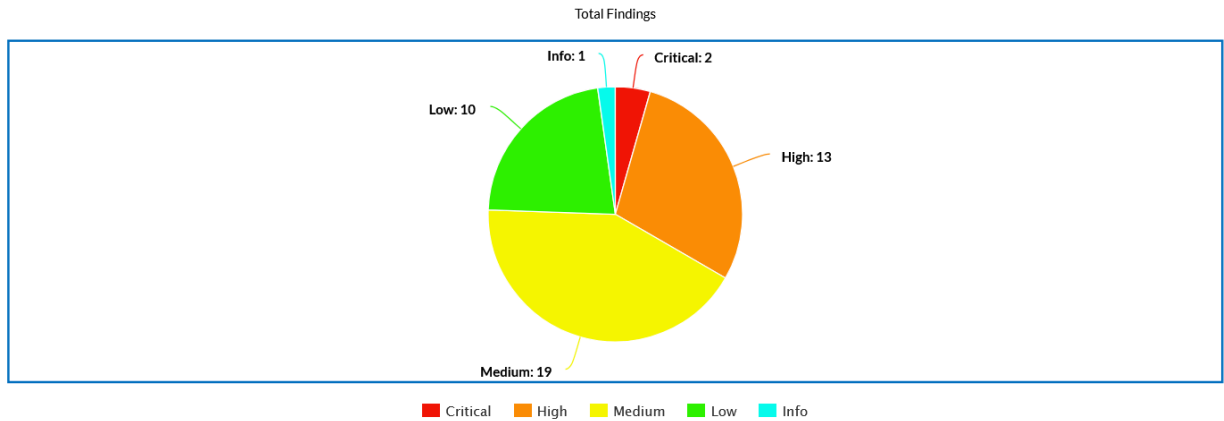
1.3 Risk Summary

- The security test reveals several **Critical** security risks that can lead to access **Admin privileges** through SQL injection and vertical privilege escalation vulnerabilities.

Additionally, **High-risk** Logic flaws vulnerabilities leads to direct **Financial Loss** through quantity tampering.

- A total of 45 vulnerabilities has been identified across 27 functionalities during the pentesting duration
 - Critical: 2
 - High: 13
 - Medium: 19
 - Low: 10

- Informational: 1



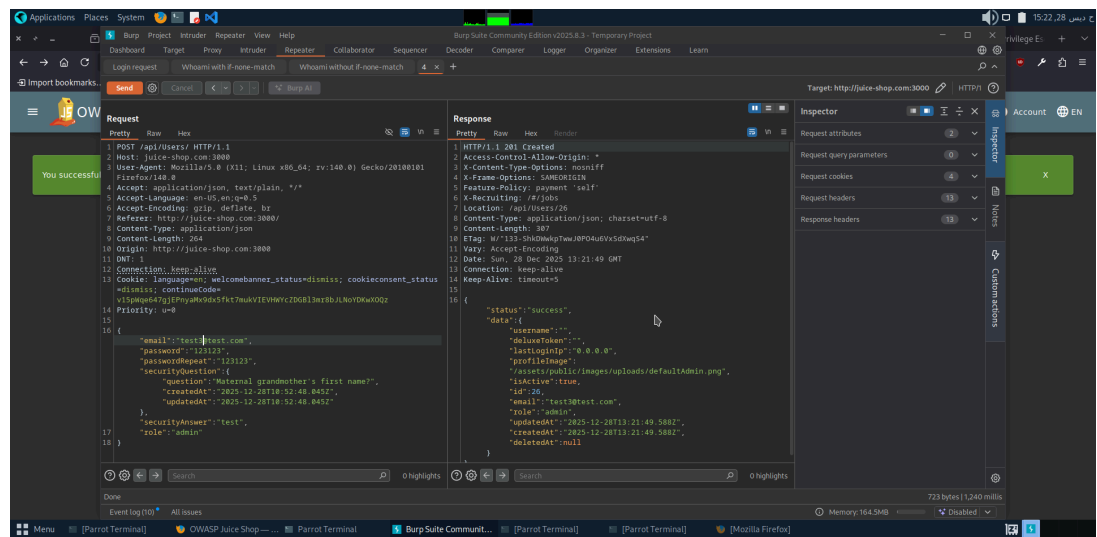
1.4 Strategic Recommendations

- To improve the security posture of the application we recommend the following measures:
 1. **Implement centralized input validation:** Move the input validation from the client-side to a robust server-side validation to prevent Injection and Logic Flaws.
 2. **Enforce the principle of least privileges:** Redesign the registration process and the role management to ensure the least Privileges are assigned to the users.
 3. **Secure the software development lifecycle:** Integrate automated security scanners (SAST/DAST) to catch the verbose error messages and the hardcoded credentials before deployment.

2. Vulnerability Write-Ups

2.1 A01:2025 - Broken Access Control

1. **Title:** Privilege escalation by adding the ("role":"admin") during registration process
 - **Risk Rating:** CVSS v4.0 Score: 9.3 / Critical
 - **Summary:** A privilege escalation was identified in the registration request through changing the "role" parameter to admin. This could allow the attacker to register an account with administrative privileges which compromises the integrity of the application.
 - **Background:** A privilege escalation vulnerability is when an attacker can use a logic flaw to elevate their privileges either vertically (higher privilege) or horizontally (other account with the same privilege).
 - **Technical Details & Evidence:**
 - **Evidence**
 - Privilege escalation by adding the "role":"admin" field to the JSON body of the registration request using burp suite (community edition is sufficient).



- Payload: "role":"admin"
- Endpoint: POST /api/Users/

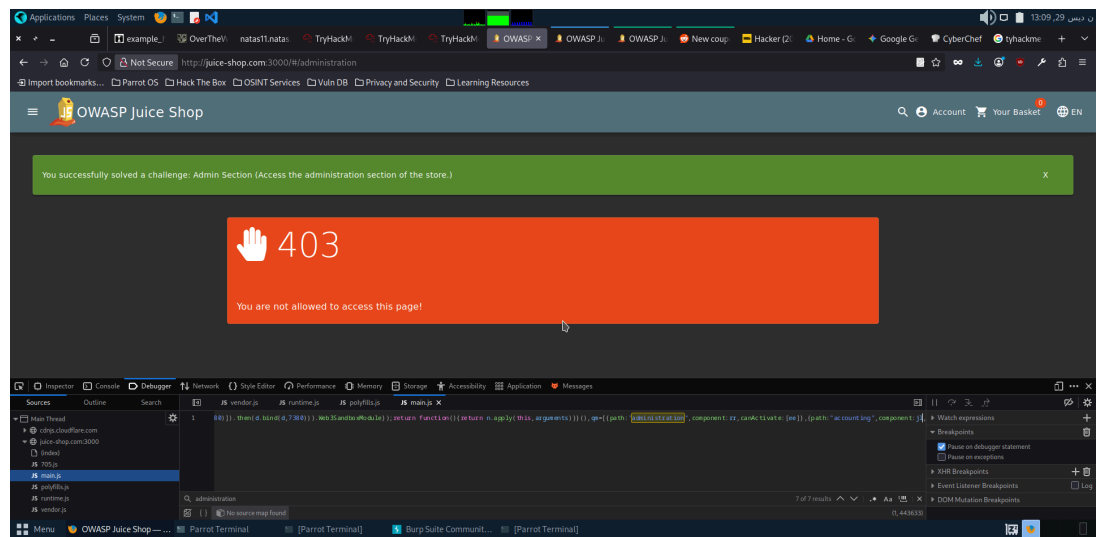
- **Reproducing steps**

1. Fill the registration form with corresponding data
2. Intercept the request using a proxy like burp suite
3. Add to the body of the request the "role" parameter with the value of "admin"
4. Forward the request

- **Impact:** This can lead unauthorized access to administrative functionalities which allow the attacker tamper with the users data and compromises the confidentiality and the integrity of the web application
- **Remediation Advice:** The registration API should only accept essential user data (email, password). The `role` should be hardcoded to `Customer` or `User` in the backend logic for all public registration endpoints.

2. Title: Administration panel path exposed in the client-side code

- **Risk Rating:** CVSS v4.0 Score: 8.6 / High
- **Summary:** Administration panel path is exposed in the client-side code in the javascript files. This can lead the attacker to use administrative functionality if paired with another vulnerability the elevate his privilege to admin which compromises the confidentiality and integrity of the data.
- **Background:** Sensitive information leakage can happens due to forgotten endpoints and credentials in the client-side code which can be inspected through the browser.
- **Technical Details & Evidence:**
 - **Evidence**
 - Administration panel path exposed in the main.js file



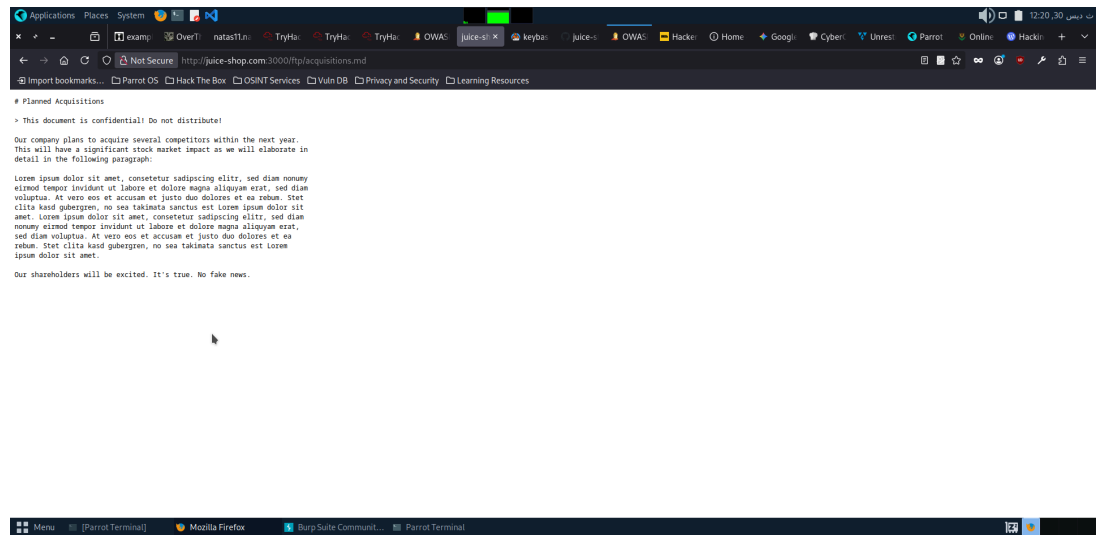
- **Reproducing steps**

1. Login to the application with any account
2. Right click any where in the browser and choose "inspect"
3. Go to debugger tab > main.js
4. Use "Ctrl+F" for searching in the file and search for admin
5. Go through the result until you hit "Path administration panel"

- **Impact:** This finding can be chained with other findings like "privilege escalation via registration" and can lead to a full control of the application features, like deleting users data and access their PII (personal identifiable information) data.
- **Remediation Advice:** Implementation of an automatic scanner (SAST/DAST) can help catching any forgotten endpoints or credentials before deployment.

3. **Title:** Sensitive data exposure through the /ftp/ endpoint

- **Risk Rating:** CVSS v4.0 Score: 8.7 / High
- **Summary:** A sensitive data is exposed through a forgotten endpoint (/ftp/). This can leads to unauthorized access to confidential data of the company.
- **Background:** Sensitive information exposure can happens due to forgotten endpoints in the deployment.
- **Technical Details & Evidence:**
 - **Evidence**
 - Sensitive information exposure through an exposed endpoint "/ftp/" that allows download of any ".md/.pdf" file



- Endpoint: <http://juice-shop.com:3000/ftp/acquisitions.md>

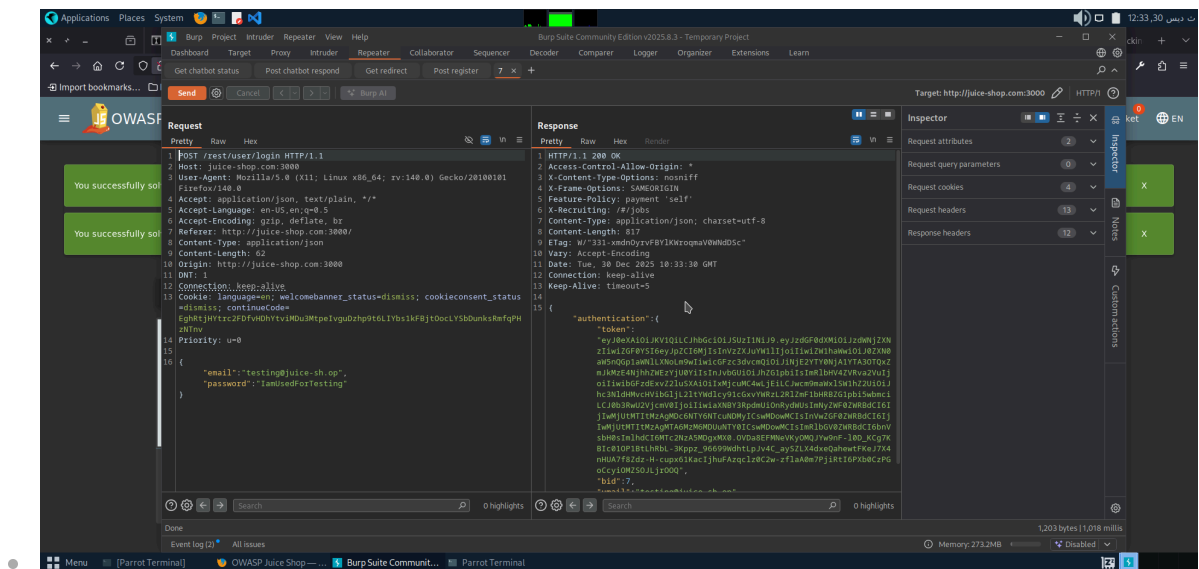
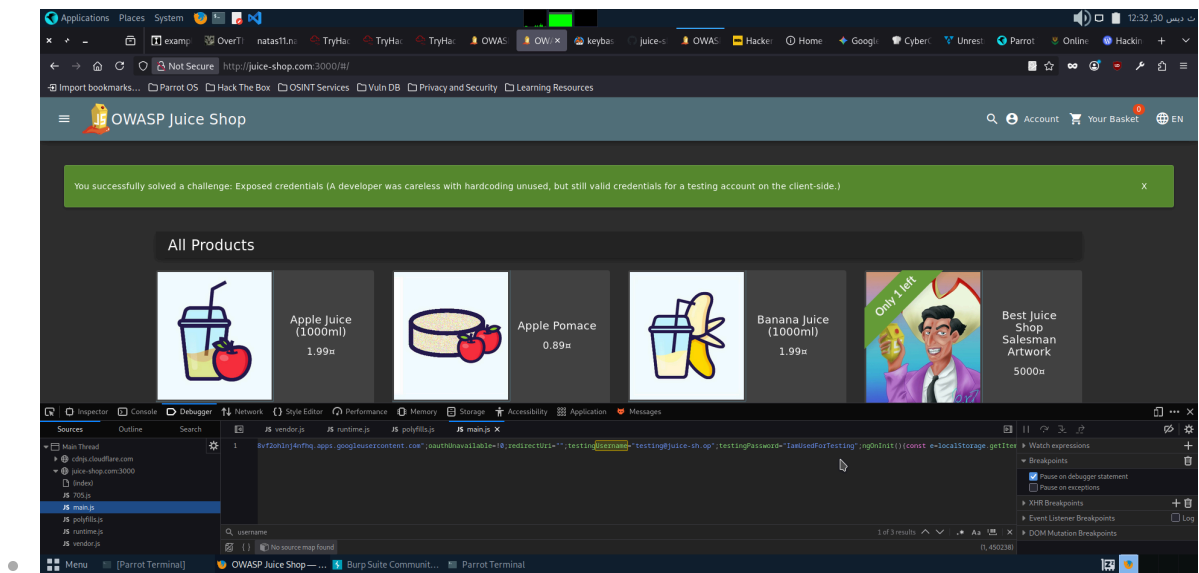
- **Reproducing steps**

1. Copy the application URL
2. Open the terminal and make sure "dirsearch" or a similar tool is installed by typing the tool name and hitting "enter"
3. Then use the command "dirsearch -u URL"
4. Access the exposed "/ftp/" endpoint in the browser

- **Impact:** Sensitive data exposure can lead to unauthorized access to confidential information of the company can cause reputational damage and losing customer trust causing indirect financial loss.
- **Remediation Advice:** Don't host sensitive information or directories in the web server, if the file is not in the web root it cannot be requested via browser.

4. **Title:** Exposed valid testing credentials in the client-side code

- **Risk Rating:** CVSS v4.0 Score: 7.2 / High
- **Summary:** A valid testing credentials was exposed in the client-side javascript files with administrative privileges. It can lead to the attacker of gaining unauthorized access to the account with higher privileges which can compromises the confidentiality and integrity of the web application.
- **Background:** Sensitive information leakage can happens due to forgotten endpoints and credentials in the client-side code which can be inspected through the browser.
- **Technical Details & Evidence:**
 - **Evidence**
 - Exposed testing credentials that still valid in the client side javascript files



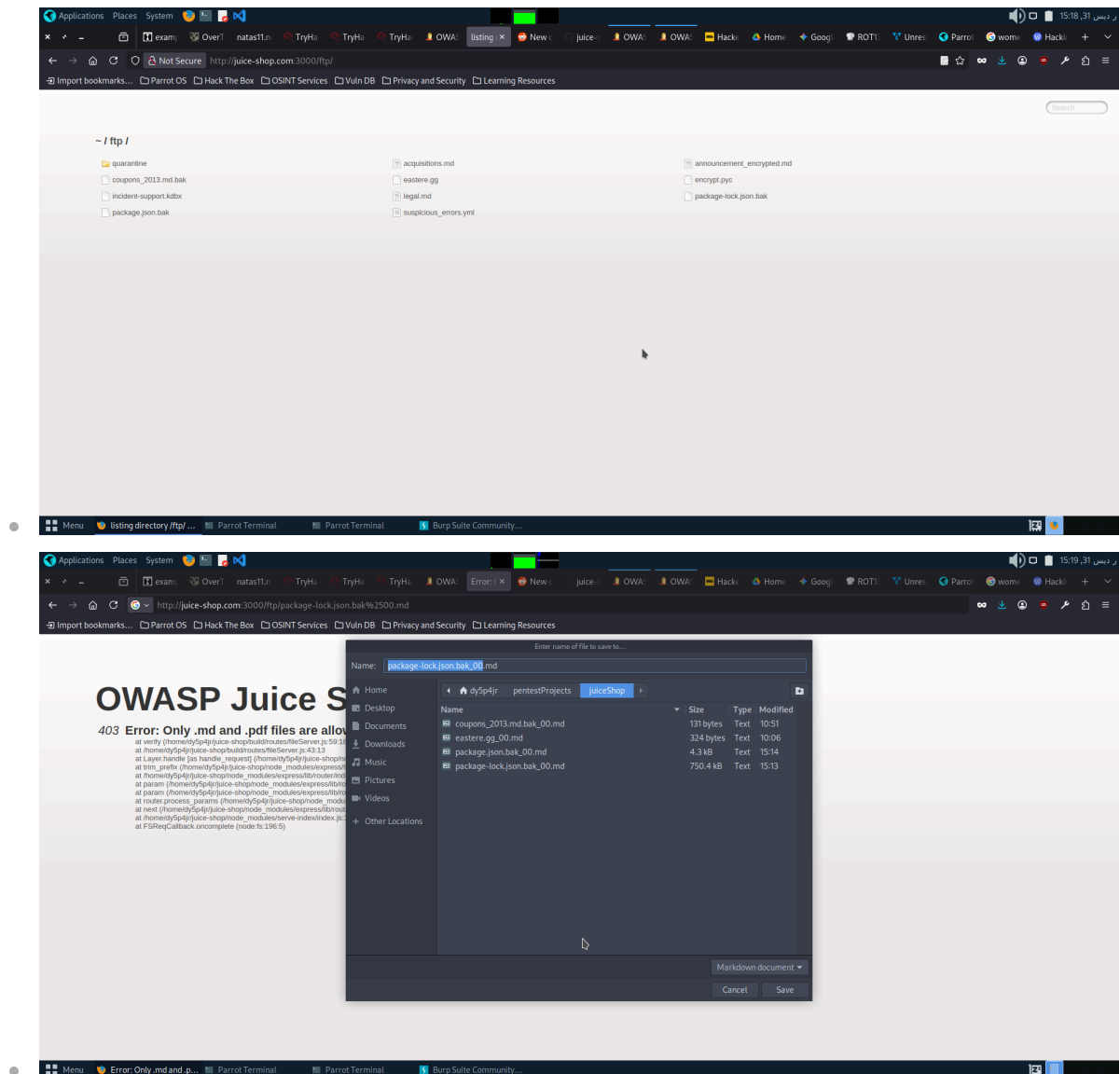
- Endpoint: <http://juice-shop.com:3000/>
- File: main.js
- **Reproducing steps**
 1. Login to the application with any account
 2. Right click any where in the browser and choose "inspect"
 3. Go to debugger > main.js
 4. Press "Ctrl+F" to search in the file and search for "Username"
 5. Go through the results until you see "testingUsername" and "testingPassword"
 6. Copy the credentials and use them to log in
 7. Navigate to the administration panel
- **Impact:** An exposed credentials with administrative privileges can cause massive harm due to the attacker ability to gain unauthorized access to administration functionality, exposing the users PII (personal identifiable information) data and take control over that data.
- **Remediation Advice:** Implementation of an automatic scanner (SAST/DAST) can help catching any forgotten endpoints or credentials before deployment.

5. Title: Backup files exposed and access is possible through null byte injection

- **Risk Rating:** CVSS v4.0 Score: 8.7 / High
- **Summary:** Backup files are exposed and can be accessed through null byte injection in the (/ftp/) endpoint. which can lead to unauthorized access to a version of the source code of the application.
- **Background:** Sensitive information leakage can happens due to forgotten endpoints and credentials in the client-side code which can be inspected through the browser.
- **Technical Details & Evidence:**

- **Evidence**

- Backup files exposed in the /ftp/ directory can be accessed using a null byte injection "%00" (the % needs to be URL encoded)



- Payload: `package-lock.json.bak%2500.md`
- Endpoints: GET `/ftp/package-lock.json.bak%2500.md`
GET `/ftp/package.json.bak%2500.md`

- **Reproducing steps**

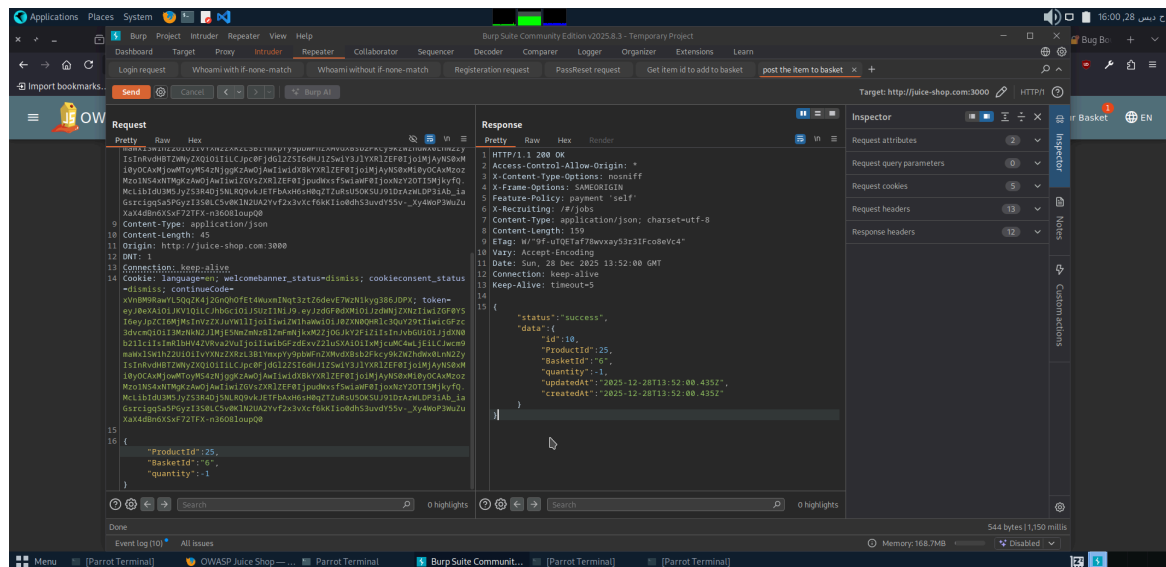
1. Access the "/ftp/" endpoint
2. Choose any file ".bak", it will redirect you to unauthorized page "403"
3. Inject the following payload in the URL at the end "%2500.md"
4. Download the backup file successfully

- **Impact:** Backup file unauthorized access can lead to the leakage of the web application information which might include vulnerabilities in the source code and can give an access to users confidential data.
- **Remediation Advice:** Don't host sensitive information or directories in the web server, if the file is not in the web root it cannot be requested via browser.

6. Title: Business logic flaw by inserting negative value of the quantity parameter

- **Risk Rating:** CVSS v4.0 Score: 8.8 / High
- **Summary:** A business logic flaw exist in the "add to basket" functionality that allows an attacker to add an item with a negative quantity. This can lead to a direct financial loss.
- **Background:** Business logic flaws occur when an application's design or implementation allows an attacker to manipulate the intended workflow of the system.
- **Technical Details & Evidence:**
 - **Evidence**

- Logic error due to lack of input validation caused by inserting negative value in the quantity parameter in the post request which leads to financial loss



- Payload: "quantity": -1
- Endpoint: POST /api/BasketItems/
- **Reproducing steps**
 1. Log into the application with any account
 2. Open a proxy like burp suite and make sure "intercept is on"
 3. Click add to basket and intercept the request

4. Navigate to the body of the request where the JSON data exist
5. Change the quantity to any negative value "e.g. -1"
6. Forward the request and turn the intercept off
7. Go through the checkout process normally
8. See that money is added to your wallet

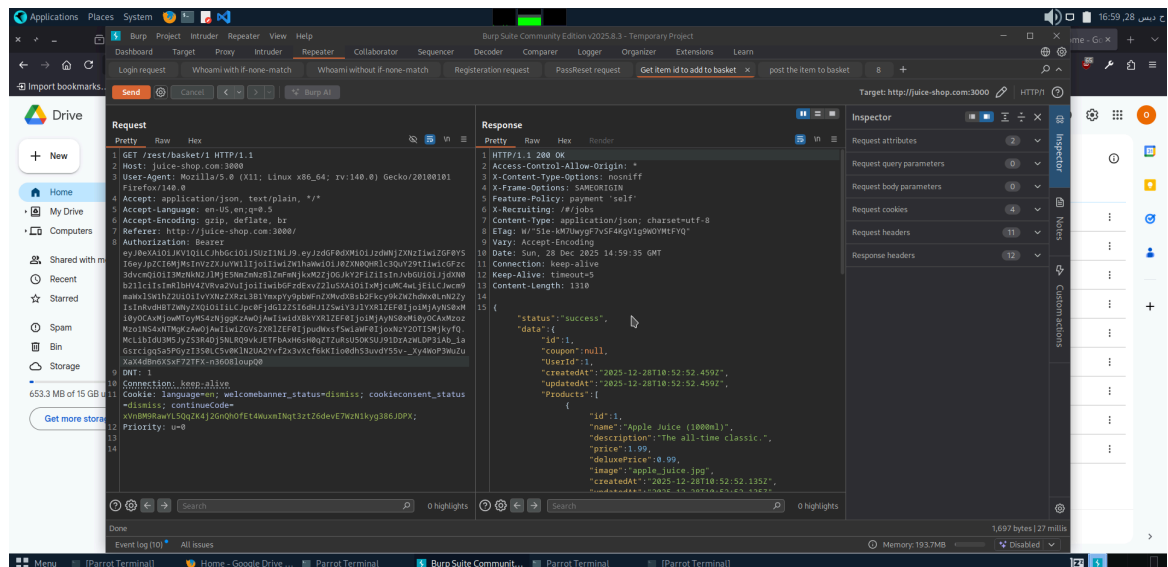
- **Impact:** It leads to direct financial loss due to processing of the request normally, which if abused massively can take the business out.
- **Remediation Advice:** Ensure that server-side validation is implemented for the user input and assume no-trust of the user input.

7. Title: IDOR leads to unauthorized access to other users baskets

- **Risk Rating:** CVSS v4.0 Score: 8.4 / High
- **Summary:** An IDOR was identified in the (GET /rest/basket/[id to access]) endpoint that leads to unauthorized access to other users baskets. Which compromises the confidentiality of other users data.
- **Background:** An IDOR vulnerability is when an attacker can gain unauthorized access to other users data through changing the id in the request.
- **Technical Details & Evidence:**

- **Evidence**

- IDOR vulnerability identified that leads to unauthorized access to other users's baskets



- Payload: GET /rest/basket/[value to access]
- Endpoint: GET /rest/basket/[id]

- **Reproducing steps**

1. Log into the application with any account
2. Open a proxy like burp suite and make sure "intercept is on"

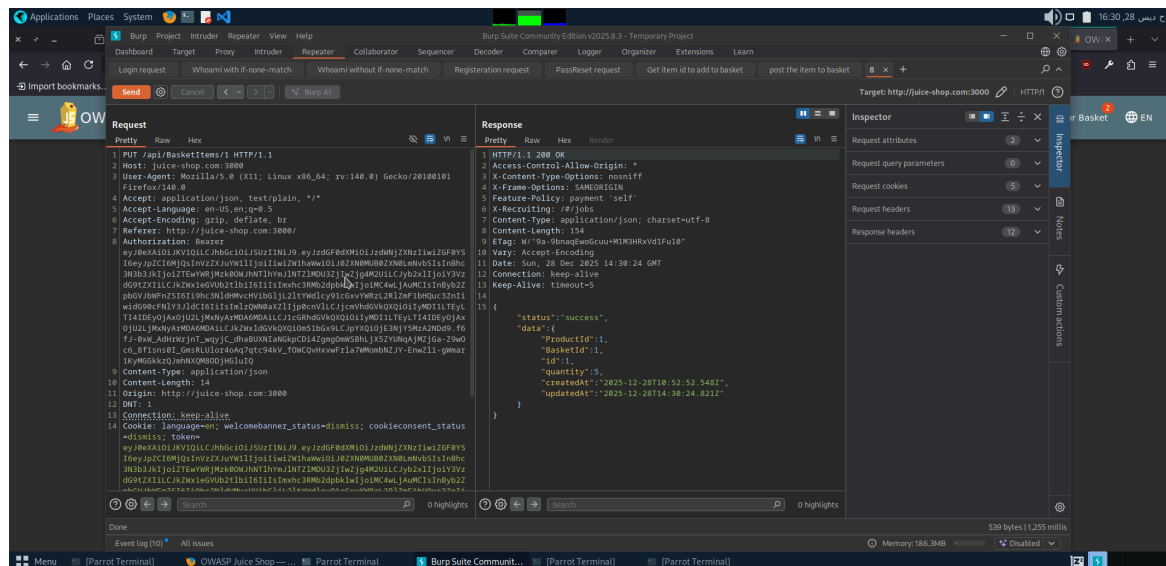
3. Click "Add to basket" on any item and intercept the request
4. Find the "GET /rest/basket/[id]" request and change the "id" to any other number (it's either gonna return an error or a success depends on if the id exists)
5. Go to the basket and see that items of other users basket is in yours

- **Impact:** This violates the user privacy and the confidentiality of the data, which can cause reputational damage to the company and indirect financial loss
- **Remediation Advice:** Ensure that server-side validation is implemented for the user input and assume no-trust of the user input. Also ensure that a "least privilege policy" is implemented and enforce it the server-side to prevent unauthorized access to others data.

8. Title: IDOR leads to adding items to other users baskets

- **Risk Rating:** CVSS v4.0 Score: 8.4 / High
- **Summary:** An IDOR vulnerability was identified the (PUT /api/BasketItems/[id to access]) endpoint that can lead the attacker to add items in other users baskets. This can lead to a direct violation of the users privacy and the integrity of the web application.
- **Background:** An IDOR vulnerability is when an attacker can gain unauthorized access to other users data through changing the id in the request.
- **Technical Details & Evidence:**

- **Evidence**
 - IDOR vulnerability leads to adding items to other users's baskets



- Payload: PUT /api/BasketItems/[id to access]
- Endpoint: PUT /api/BasketItems/[id]
- **Reproducing steps**
 1. Log into the application with any account
 2. Open a proxy like burp suite and make sure "intercept is on"
 3. Click "Add to basket" on any item and intercept the request

- **Impact:** This violates the user privacy and the confidentiality of the data, which can cause reputational damage to the company and indirect financial loss.
- **Remediation Advice:** Ensure that server-side validation is implemented for the user input and assume no-trust of the user input. Also ensure that a "least privilege policy" is implemented and enforce it the server-side to prevent unauthorized access to others data.

- **Risk Rating:** CVSS v4.0 Score: 8.4 / High
- **Summary:** An IDOR vulnerability was identified the (DELETE /api/BasketItems/[item id to delete]) endpoint that can lead the attacker to delete other users items. This is a direct violation to the privacy of other users data and compromises the web application confidentiality
- **Background:** An IDOR vulnerability is when an attacker can gain unauthorized access to other users data through changing the id in the request.

- **Evidence**

- The screenshot displays the Burp Suite Community Edition interface. The top bar shows the application name and version (v2025.8.3 - Temporary Project). The main window is divided into several panes. The left pane shows a list of requests, with the first one selected. The middle pane shows the details of the selected request, including the raw HTTP text and the response. The right pane shows the response details, including the status code (200 OK) and the response body (a JSON object indicating success). The interface is in English, and the target is set to http://paulce-shop.com:3000.

- **Reproducing steps**

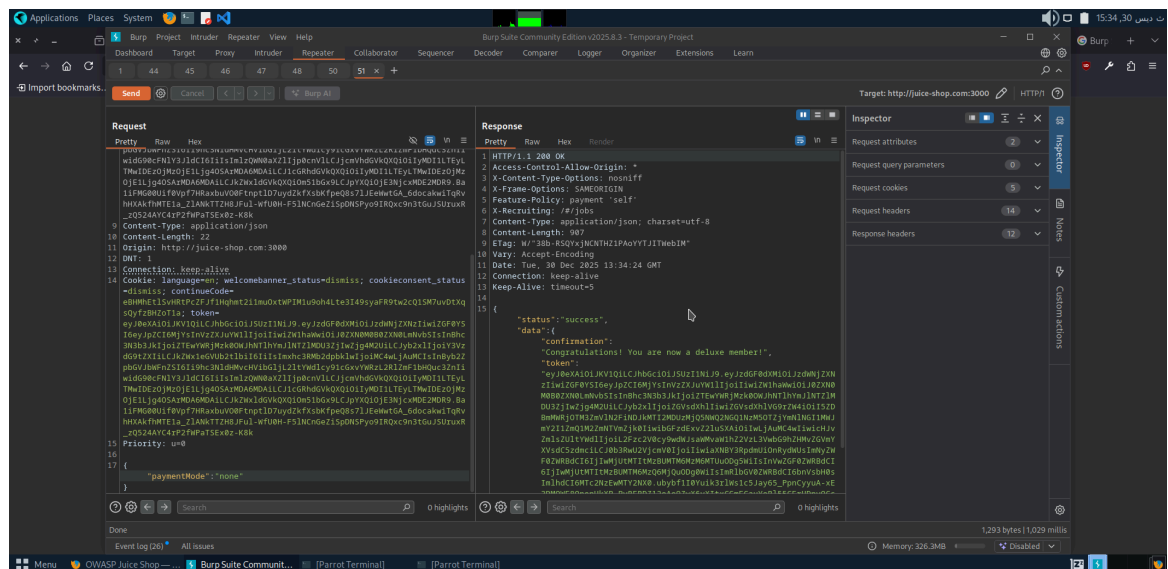
1. Log into the application with any account

2. Click "Add to basket" on any item and intercept the request
3. Open a proxy like burp suite and make sure "intercept is on"
4. Click the "delete" button and intercept the request
5. Find the "DELETE /api/BasketItems/[id]" and change the "id" to other user's basket "id" (use a second account and go through the steps to to capture the request and view the "basket id")
6. Navigate to the other account basket and realize the item is deleted

- **Impact:** This violates the user privacy and the confidentiality of the data, which can cause reputational damage to the company and indirect financial loss.
- **Remediation Advice:** Ensure that server-side validation is implemented for the user input and assume no-trust of the user input. Also ensure that a "least privilege policy" is implemented and enforce it the server-side to prevent unauthorized access to others data.

10. **Title:** Business logic flaw leads to submit "none" as a payment method in the membership functionality

- **Risk Rating:** CVSS v4.0 Score: 7.1 / High
- **Summary:** A business logic flaw leads to bypass the payment of the membership and gain unauthorized membership through modifying the payment method to "none" in the request body. This leads to unauthorized access of the membership features without the payment for it which a direct financial loss to the business.
- **Background:** Business logic flaws occur when an application's design or implementation allows an attacker to manipulate the intended workflow of the system.
- **Technical Details & Evidence:**
 - **Evidence**
 - Lack of input validation in the server side leads to submit "none" as a payment method resulting in getting the membership without paying any money



- **Payload:** "paymentMode":"none"

- Endpoint: POST /rest/deluxe-membership

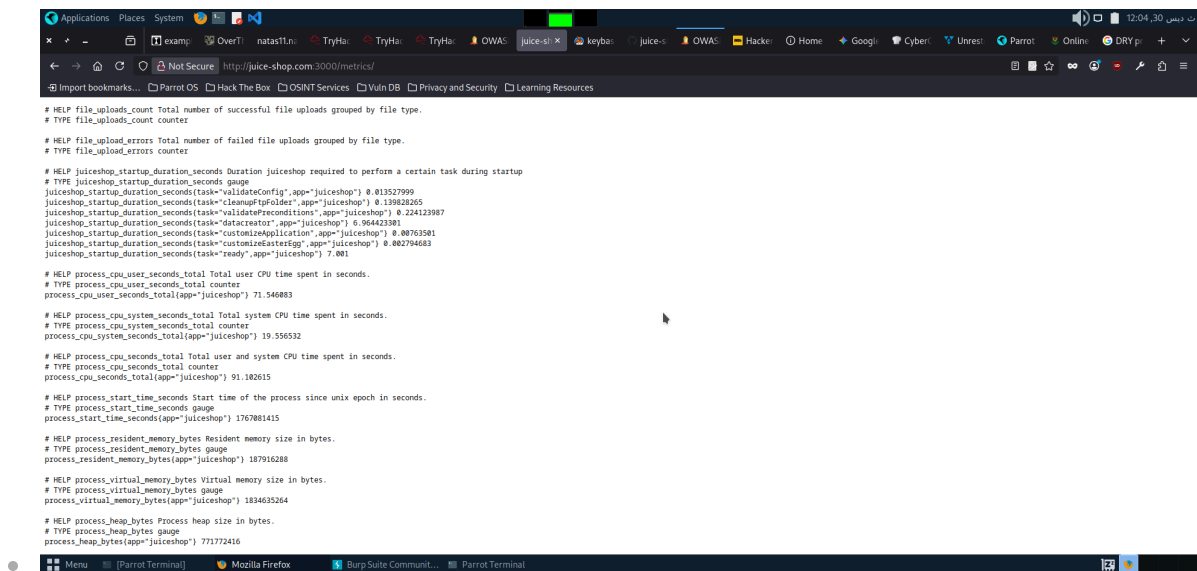
- **Reproducing steps**

1. Log into any account
2. Navigate to the membership page
3. Click "Become a member" button
4. Open any proxy like burp suite and make sure "intercept is on"
5. Add your card details or choose to pay from wallet and intercept the confirm request
6. Find "POST /rest/deluxe-membership" endpoint and go to the body where the JSON data exist
7. Change the "paymentMode" parameter to "none" and remove any other parameters
8. Forward the request and see that you became a member without paying

- **Impact:** This is a direct financial loss allowing the attacker to use membership features without paying for it.
- **Remediation Advice:** Ensure that server-side validation is implemented for the user input and assume no-trust of the user input.

11. **Title:** Sensitive information leakage of the server metrics through the /metrics endpoint

- **Risk Rating:** CVSS v4.0 Score: 6.9 / Medium
- **Summary:** A forgotten endpoint "/metrics" leaks the server metrics data without unauthorized access, which leads to sensitive data regarding the server infrastructure and performance to be accessed by an attacker.
- **Background:** Sensitive information exposure can happens due to forgotten endpoints in the deployment.
- **Technical Details & Evidence:**
 - **Evidence**
 - Sensitive information leakage through exposed metrics endpoint



```
# HELP file_uploads_count Total number of successful file uploads grouped by file type.
# TYPE file_uploads_count counter

# HELP file_upload_errors Total number of failed file uploads grouped by file type.
# TYPE file_upload_errors counter

# HELP juiceshop_startup_duration_seconds Duration juiceshop required to perform a certain task during startup
# TYPE juiceshop_startup_duration_seconds gauge
juiceshop_startup_duration_seconds(task="validateConfig",app="juiceshop") 0.013527999
juiceshop_startup_duration_seconds(task="cleanupTmpFolder",app="juiceshop") 0.13962305
juiceshop_startup_duration_seconds(task="validatePreconditions",app="juiceshop") 0.224123987
juiceshop_startup_duration_seconds(task="datacreator",app="juiceshop") 0.96423381
juiceshop_startup_duration_seconds(task="customizeApplication",app="juiceshop") 0.00763581
juiceshop_startup_duration_seconds(task="customizeEasterEgg",app="juiceshop") 0.002794683
juiceshop_startup_duration_seconds(task="ready",app="juiceshop") 7.001

# HELP process_cpu_user_seconds_total Total user CPU time spent in seconds.
# TYPE process_cpu_user_seconds_total counter
process_cpu_user_seconds_total(app="juiceshop") 71.546883

# HELP process_cpu_system_seconds_total Total system CPU time spent in seconds.
# TYPE process_cpu_system_seconds_total counter
process_cpu_system_seconds_total(app="juiceshop") 19.556532

# HELP process_cpu_seconds_total Total user and system CPU time spent in seconds.
# TYPE process_cpu_seconds_total counter
process_cpu_seconds_total(app="juiceshop") 91.103415

# HELP process_start_time_seconds Start time of the process since unix epoch in seconds.
# TYPE process_start_time_seconds gauge
process_start_time_seconds(app="juiceshop") 1767801415

# HELP process_resident_memory_bytes Resident memory size in bytes.
# TYPE process_resident_memory_bytes gauge
process_resident_memory_bytes(app="juiceshop") 187916288

# HELP process_virtual_memory_bytes Virtual memory size in bytes.
# TYPE process_virtual_memory_bytes gauge
process_virtual_memory_bytes(app="juiceshop") 1834635264

# HELP process_heap_bytes Process heap size in bytes.
# TYPE process_heap_bytes gauge
process_heap_bytes(app="juiceshop") 77372416
```

- Endpoint: <http://juice-shop.com:3000/metrics/>

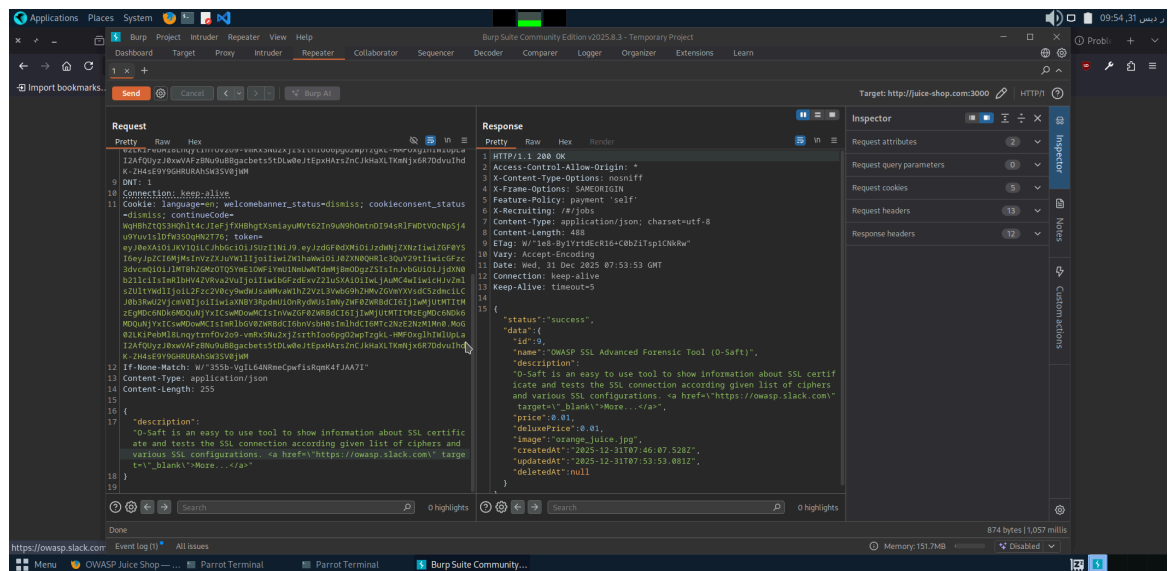
- **Reproducing steps**

1. Copy the web application URL
2. Make sure you have dirsearch or a similar tool installed on the system
3. Open the terminal and write the following command "dirsearch -u URL"
4. Access the exposed "/metrics" endpoint in the browser

- **Impact:** Gaining access to information regarding the server hardware can show a weak point for a DoS (Denial of server attack), also the metrics leak other users' data and provide a deeper look at the technology stack of the server (helps to find any known vulnerabilities).
- **Remediation Advice:** Don't host sensitive information or directories in the web server, if the file is not in the web root it cannot be requested via browser.

12. Title: Product tampering in the products API

- **Risk Rating:** CVSS v4.0 Score: 5.1 / Medium
- **Summary:** A product tampering is possible through the products API with just changing the product's data in the JSON body, which leads to changing the price of a product and other data manipulation damaging the company reputation and potential financial loss.
- **Background:** Business logic flaws occur when an application's design or implementation allows an attacker to manipulate the intended workflow of the system.
- **Technical Details & Evidence:**
 - **Evidence**
 - Lack of server-side validation leads to product tampering and can lead to inject malicious links to redirect users to



- Payload: "description": "More..."
- Endpoint: PUT /api/products/9

• Reproducing steps

1. Log in with any account
2. Open any proxy like burp suite and make sure that "intercept is on"
3. Click on "Add to basket" and intercept the request
4. Find any endpoint like "POST /api/BasketItems/" and change right click it and choose "send to repeater"
5. Go to the "repeater" tab and change the request to "PUT /api/products/[product id]"
6. Send the request and see in the response body the list of the parameters
7. Copy the parameter you want to change to the request JSON body and change it to any value you want (including embedded links in the description)
8. Send the request and click on the product (make sure you turn the intercept off) and see the changes to the products

- **Impact:** The impact is potential financial loss and damaging to the reputation, also it's possible to embed links in the products description which might redirect the user a malicious domain controlled by the attacker
- **Remediation Advice:** Ensure that server-side validation is implemented for the user input and assume no-trust of the user input.

13. **Title:** Business logic flaw leads to submit negative or exceed the positive value of rating in the feedback functionality

- **Risk Rating:** CVSS v4.0 Score: 5.3 / Medium
- **Summary:** A logic flaw is present in the feedback functionality leads to submit negative or exceeds the positive value of the rating. This can cause damage to the reputation of the

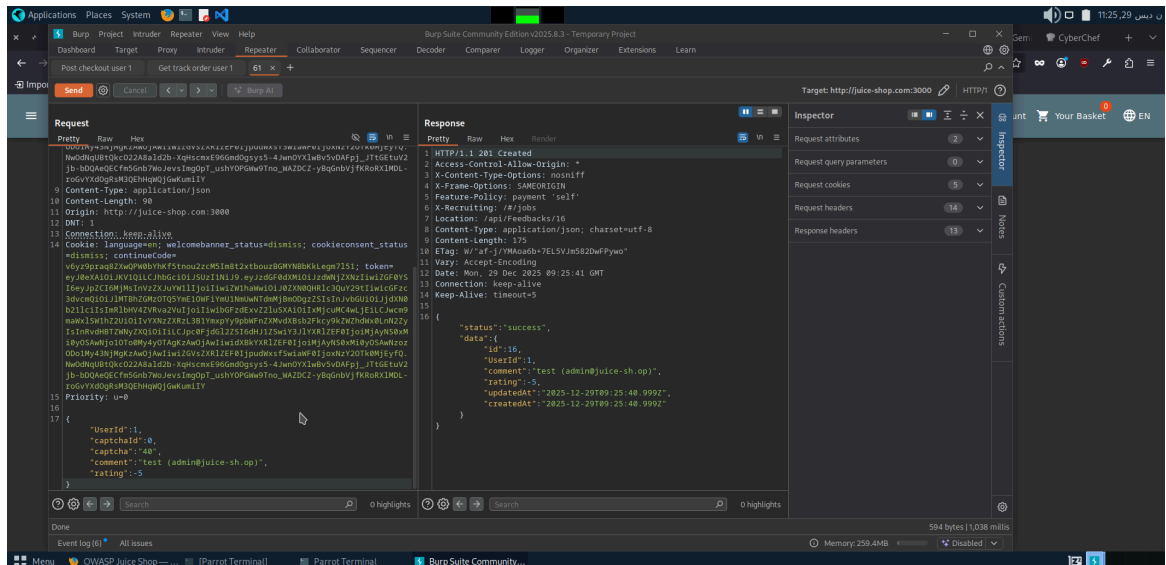
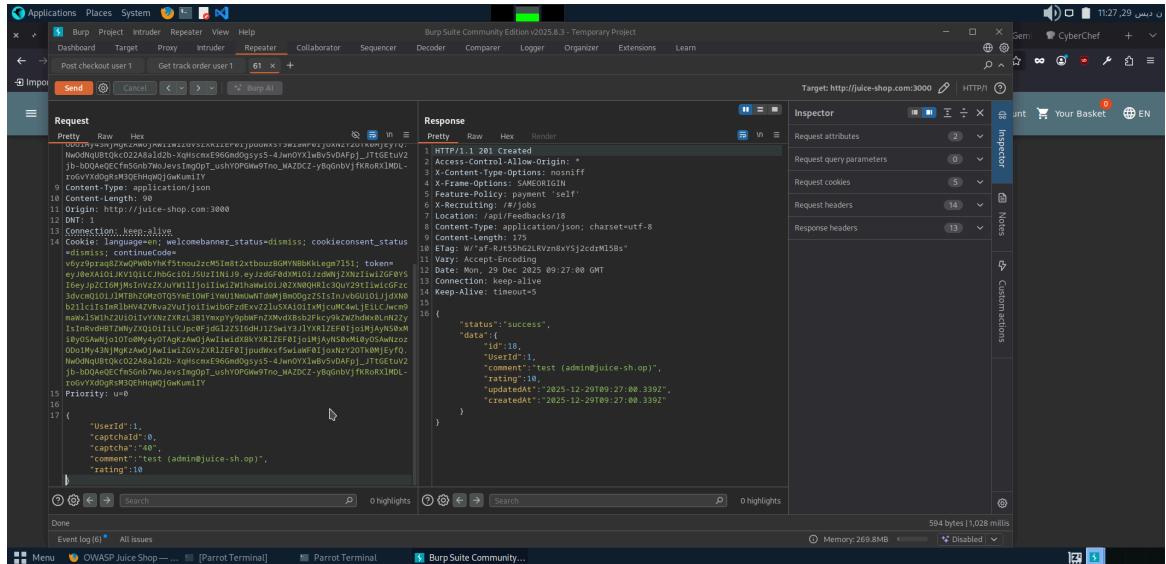
company and clobber the server database with incorrect data.

- **Background:** Business logic flaws occur when an application's design or implementation allows an attacker to manipulate the intended workflow of the system.

- **Technical Details & Evidence:**

- **Evidence**

- Lack of json data parameters validation leads to submit negative or exceed the positive value of rating



- Payload: "rating":10/-10
 - Endpoint: POST /api/Feedbacks/

- **Reproducing steps**

1. Log in with any account and navigate to the feedback page
2. Open a web proxy like burp suite and make sure that "intercept is on"
3. Fill in the form with any data and click submit to intercept the request
4. Find the "POST /api/Feedbacks/" endpoint and right click it and choose "send to repeater" then navigate to the repeater tab

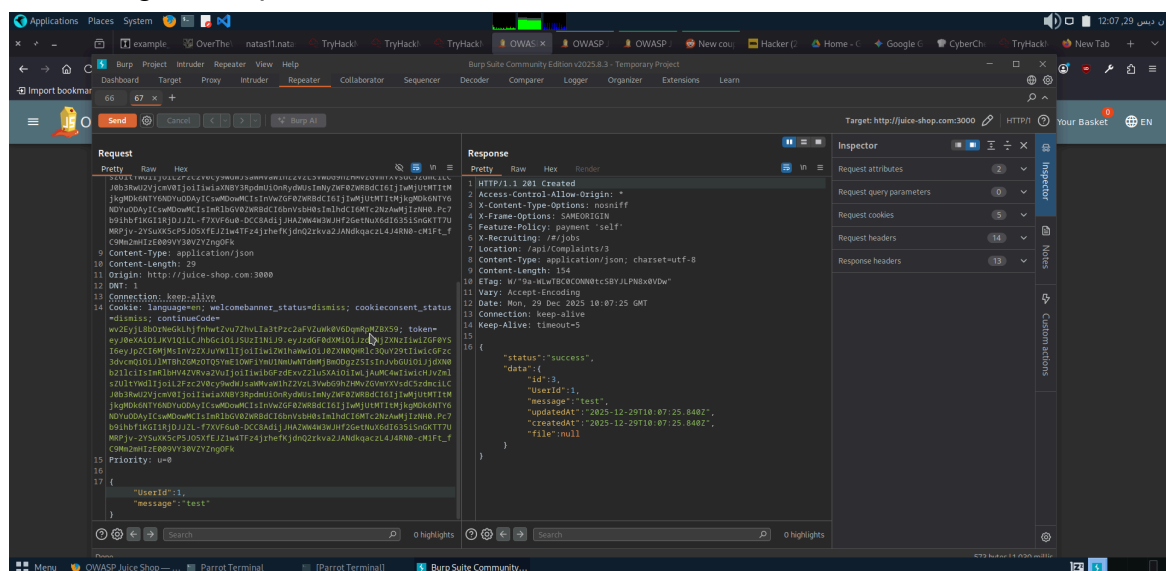
5. Change the "rating" parameter in request JSON body data to "-10" (or 10 or any numeric value you want) and click send

6. Notice that the request is send successfully

- **Impact:** A reputational damage is a direct harm caused by this vulnerability which can lead to indirect financial loss, and side effect is clobbering the server database with incorrect data that can consume time to clean and gives incorrect representation of the business performance.
- **Remediation Advice:** Ensure that server-side validation is implemented for the user input and assume no-trust of the user input.

14. **Title:** Submitting complains on behalf of other users in the customer support form (complain)

- **Risk Rating:** CVSS v4.0 Score: 5.1 / Medium
- **Summary:** A logic flaw is possible due to trusting the client-side user input which can lead to change the email address to any email and submit a feedback on behalf of it. This can cause reputational damage and violates the integrity of the data in the web application.
- **Background:** Business logic flaws occur when an application's design or implementation allows an attacker to manipulate the intended workflow of the system.
- **Technical Details & Evidence:**
 - **Evidence**
 - Trust in the json data parameters in complaints page leads to unauthorized submitting of complains on behalf of another user



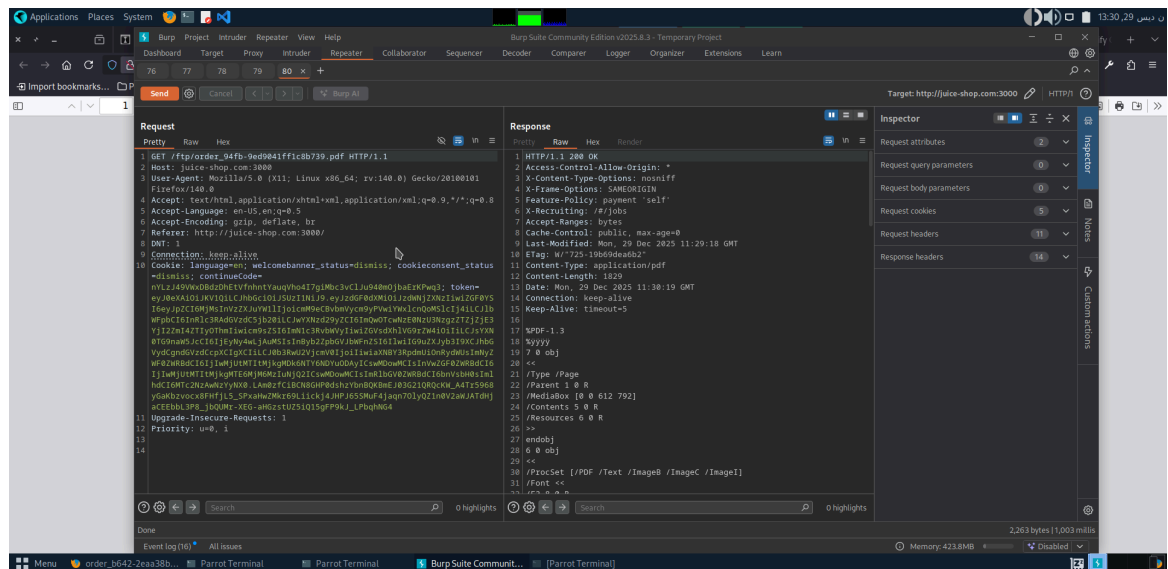
- Payload: "UserId":1
- Endpoint: POST /api/Complaints/
- **Reproducing steps**
 1. Log in with any account and navigate to the complaints page

2. Open a web proxy like burp suite and make sure that "intercept is on"
3. Fill in the form with any data and click submit to intercept the request
4. Find the "POST /api/Complaints/" endpoint and right click it and choose "send to repeater" then navigate to the repeater tab
5. Change the "UserId" parameter in request JSON body data to any user id and click send
6. Notice that the request is send successfully

- **Impact:** The potential reputational damage can lead to indirect financial loss to the business and consume the customer support team's time processing false complaints, also it's possible to clobber the database with incorrect representation of the data causing a wrong representation of the business performance and consumed time to clean up the inaccurate data.
- **Remediation Advice:** Ensure that server-side validation is implemented for the user input and assume no-trust of the user input.

15. **Title:** IDOR leads to unauthorized access to the order confirmation pdf of another user by replacing the UUID

- **Risk Rating:** CVSS v4.0 Score: 4.8 / Medium
- **Summary:** There's an IDOR vulnerability in the "GET /ftp/order_[uuid].pdf" endpoint that can lead to unauthorized access to other users order confirmation data. This can lead a confidential users data and damaging the reputation of the company.
- **Background:** An IDOR vulnerability is when an attacker can gain unauthorized access to other users data through changing the id in the request.
- **Technical Details & Evidence:**
 - **Evidence**
 - Replacing the uuid of any user allows unauthorized access to the order confirmation pdf



- Payload: GET /ftp/order_[uuid to access].pdf
- Endpoint: GET /ftp/order_[uuid].pdf

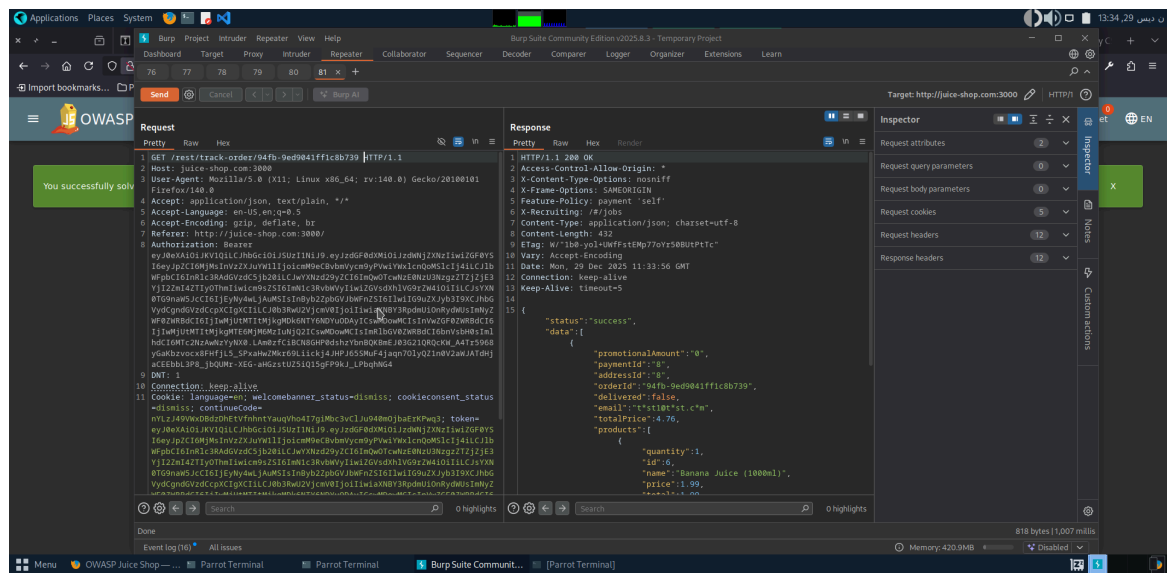
- **Reproducing steps**

1. Log in with any account
2. Open a web proxy like burp suite and make sure that "intercept is on"
3. Capture any request sent by the web application (e.g. Add to basket request) and right click it and choose "send to repeater"
4. Navigate to the repeater tab and change the request endpoint to "GET /ftp/order_[uuid].pdf" (you can make another account and make an order then capture the order confirmation request in the "order history" page)
5. Replace the UUID with any other UUID and click send
6. Notice that you can now view others order confirmation and save it

- **Impact:** Accessing other users confidential data can cause reputational damage to the company leading to indirect financial loss.
- **Remediation Advice:** Ensure that server-side validation is implemented for the user input and assume no-trust of the user input. Also ensure that a "least privilege policy" is implemented and enforce it the server-side to prevent unauthorized access to others data.

16. **Title:** IDOR leads to unauthorized access to tracking order of other users through replacing the UUID

- **Risk Rating:** CVSS v4.0 Score: 4.8 / Medium
- **Summary:** It's possible to access other users tracking order through replacing the UUID in the request (IDOR vulnerability). This consider a violation to the users data confidentiality and can cause damage to the company reputation
- **Background:** An IDOR vulnerability is when an attacker can gain unauthorized access to other users data through changing the id in the request.
- **Technical Details & Evidence:**
 - **Evidence**
 - Replacing the uuid of any user allows unauthorized tracking order of other users



- Payload: GET /rest/track-order/[uuid to access]
- Endpoint: GET /rest/track-order/[uuid]

• Reproducing steps

1. Create two test accounts
2. Log in with account 1 (the victim) and go through the process of making an order until the end of checkout
3. Copy the UUID and Log into account 2 (the attacker)
4. Open burp suite or any web proxy and make sure that "intercept is on"
5. Capture any request sent by the web application and change the endpoint to "GET /rest/track-order/[copied uuid]"
6. Click send and access the account 1 (the victim)'s data

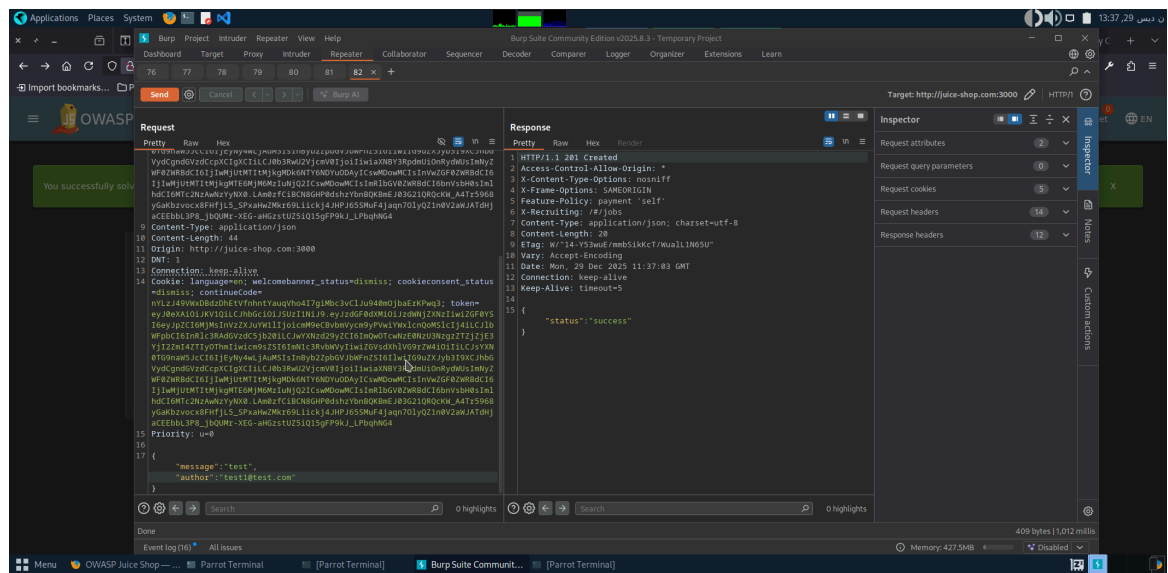
- **Impact:** Accessing other users tracking order compromises the users confidential data which leads to reputational damage and potential financial loss.
- **Remediation Advice:** Ensure that server-side validation is implemented for the user input and assume no-trust of the user input. Also ensure that a "least privilege policy" is implemented and enforce it the server-side to prevent unauthorized access to others data.

17. Title: Submitting reviews on other users behalf in the review functionality

- **Risk Rating:** CVSS v4.0 Score: 5.3 / Medium
- **Summary:** It's possible to submit reviews on behalf of other users. Which leads to compromising data integrity and can cause reputational damage.
- **Background:** Business logic flaws occur when an application's design or implementation allows an attacker to manipulate the intended workflow of the system.
- **Technical Details & Evidence:**

• Evidence

- Lack of backend validation leads to submit reviews on other users behalf



- Payload: "author": "test1@test.com"
- Endpoint: PUT /rest/products/[id]/reviews
- **Reproducing steps**
 1. Log in with any account
 2. Click on any product and add a review (but don't submit yet)
 3. Open burp suite or any web proxy of your choice and make sure "intercept is on"
 4. Click submit to capture the request
 5. Find "PUT /rest/products/[id]/reviews" endpoint and navigate to body of the request where JSON data is present
 6. Change the "author" parameter to any email of choice (e.g. admin@juice-sh.op) and forward the request
 7. Go back to the reviews and see the review is added
- **Impact:** Reputational damage is possible due to compromising the data integrity which can lead to potential financial loss and customers losing trust in the company
- **Remediation Advice:** Ensure that server-side validation is implemented for the user input and assume no-trust of the user input.

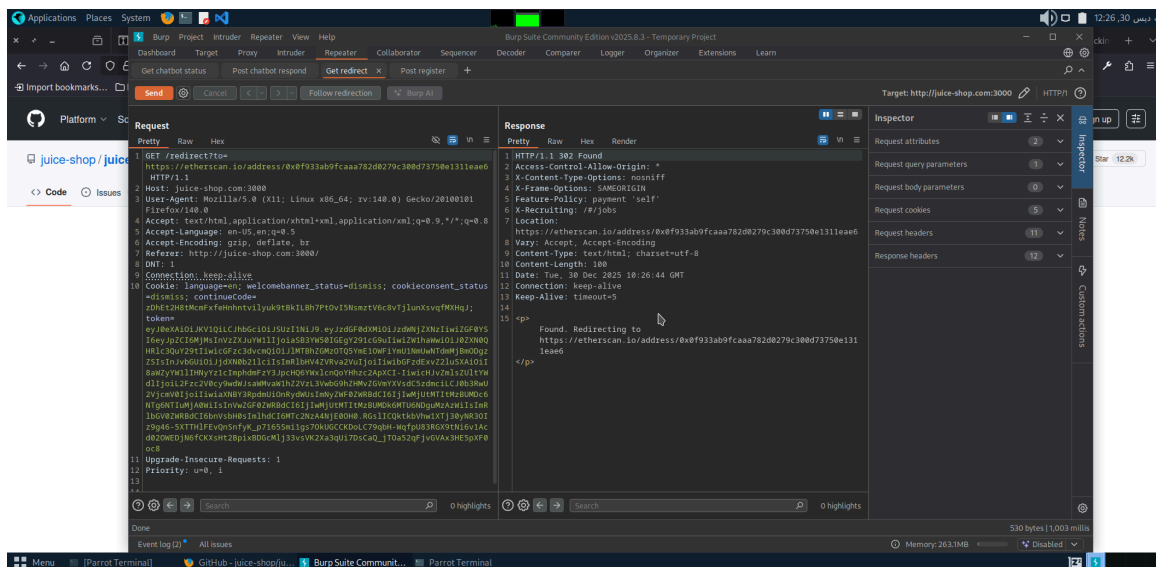
18. **Title:** Outdated URL allow list redirect in the GitHub redirect endpoint

- **Risk Rating:** CVSS v4.0 Score: 4.8 / Medium
- **Summary:** Through analysis of the web application's source code an outdated allow-list of links has been found and when testing it in the GitHub redirect endpoint it worked and a redirection successfully went through. If outdated links is no longer in use and an attacker registered such a link it can lead to redirecting the users to malicious attacker-controlled website.
- **Background:** An open redirect vulnerability happens when the web application accepts a user-controlled input that specifies a link to an external site, and uses that link in a redirect.

- **Technical Details & Evidence:**

- **Evidence**

- Outdated URL allow list redirect is identified through code review of the github repo "<https://github.com/juice-shop/juice-shop/>"



- Payload: <http://juice-shop.com:3000/redirect?to=https://etherscan.io/address/0x0f933ab9fcaaa782d0279c300d73750e1311eae6>
 - Endpoint: GET /redirect?to=

- **Reproducing steps**

1. Log in with any account
2. Open burp suite or any web proxy of your choice and make sure "intercept is on"
3. Click on the GitHub redirect page and intercept the request
4. Change the "?to=" parameter's value to any link of the outdated allow-list found in the "<https://github.com/juice-shop/juice-shop/blob/master/routes/redirect.ts>" repo and forward the request

- **Impact:** If an attacker owned one of those outdated links, it can leads to compromises the users data safety by redirecting them to an attacker-controlled malicious site (Which can run arbitrary javascript code in the user's browser or steal any headers information send along the redirection).
- **Remediation Advice:** Assume all input is malicious. Use an "accept known good" input validation strategy, i.e., use a list of acceptable inputs that strictly conform to specifications. Reject any input that does not strictly conform to specifications, or transform it into something that does.

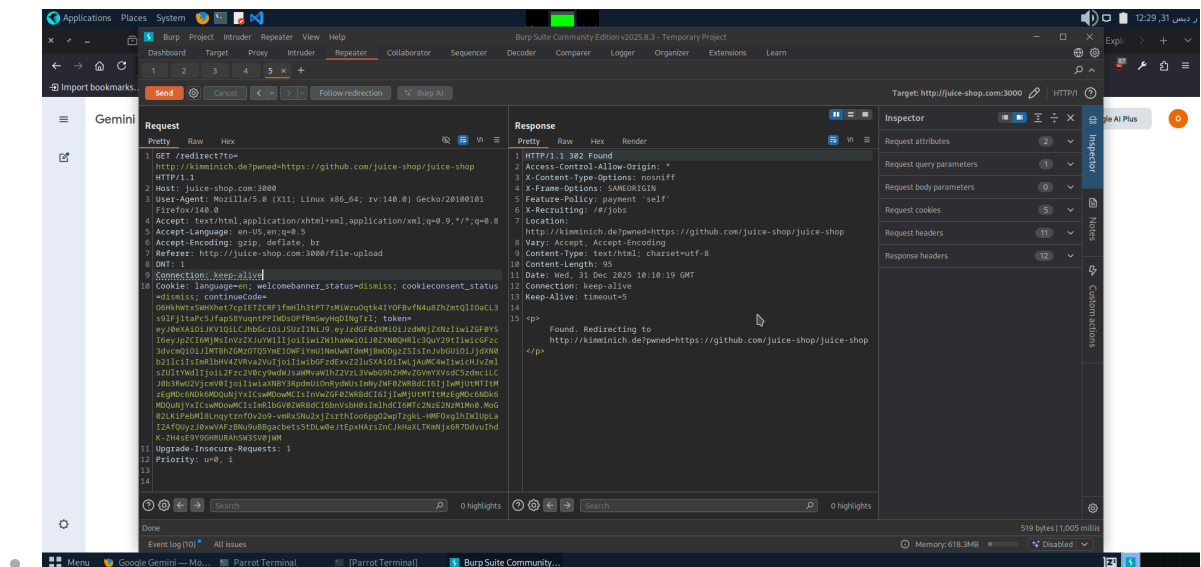
19. **Title:** Allow list redirect bypass in the GitHub redirect endpoint

- **Risk Rating:** CVSS v4.0 Score: 5.8 / Medium

- **Summary:** It's possible to bypass the redirect allow-list if the final destination is one the allow-list links, meaning it can forward the redirection to an attacker-controlled site if it then redirect the user to an allow-list link. This can compromises users data safety by redirecting the users to a malicious site it can run an arbitrary javaScript code in their browser to steal cookies or any sent headers through the redirection before redirecting them again.
- **Background:** An open redirect vulnerability happens when the web application accepts a user-controlled input that specifies a link to an external site, and uses that link in a redirect.
- **Technical Details & Evidence:**

- **Evidence**

- Allowedlist redirect bypass is possible due to improper handling of the allowedlist logic which is only checking for the final destination even if a middle redirect is present which can lead to steal user session cookies and execute javascript code in the user's browser



- Payload: <http://kimminich.de?pwd=https://github.com/juice-shop/juice-shop>
- Endpoint: GET /redirect?to=

- **Reproducing steps**

1. Log in with any account
2. Open burp suite or any web proxy of your choice and make sure "intercept is on"
3. Click on the GitHub redirect page and intercept the request
4. Change the "?to=" parameter's value to any link that then redirect again to any link of the outdated allow-list found in the "<https://github.com/juice-shop/juice-shop/blob/master/routes/redirect.ts>" repo and forward the request

- **Impact:** Redirecting a user to an attacker-controlled malicious site, can leads to compromises the users data safety by running arbitrary javaScript code in the user's browser to steal their cookies or stealing any headers information send along the redirection.

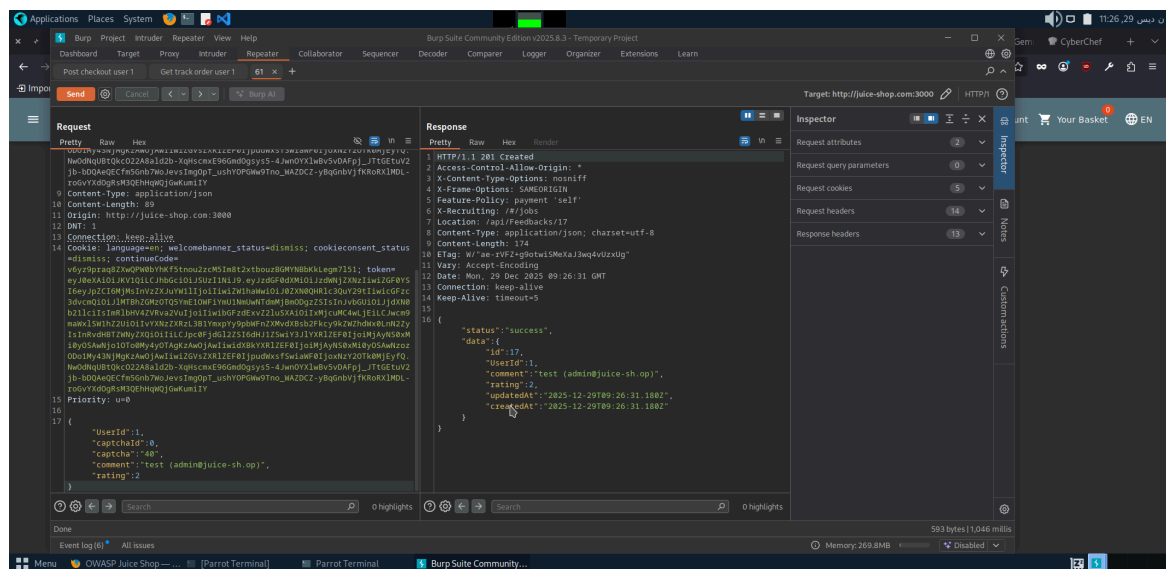
- **Remediation Advice:** Assume all input is malicious. Use an "accept known good" input validation strategy, i.e., use a list of acceptable inputs that strictly conform to specifications. Reject any input that does not strictly conform to specifications, or transform it into something that does.

20. Title: Submitting feedbacks on behalf of other users through JSON data parameters

- **Risk Rating:** CVSS v4.0 Score: 2.3 / Low
- **Summary:** A logic flaw is possible due to trusting the client-side user input which can lead to change the email address to any email and submit a feedback on behalf of it. This can cause reputational damage and violates the integrity of the data in the web application.
- **Background:** Business logic flaws occur when an application's design or implementation allows an attacker to manipulate the intended workflow of the system.
- **Technical Details & Evidence:**

- **Evidence**

- Lack of json data parameters validation leads to submit feedbacks on behalf of other users



- Payload: "UserId":1, "comment": "test (admin@juice-sh.op)"
- Endpoint: POST /api/Feedbacks/

- **Reproducing steps**

1. Log in with any account and navigate to the feedback page
2. Open a web proxy like burp suite and make sure that "intercept is on"
3. Fill in the form with any data and click submit to intercept the request
4. Find the "POST /api/Feedbacks/" endpoint and right click it and choose "send to repeater" then navigate to the repeater tab
5. Change the "UserId" parameter in request JSON body data to any user id and change the "comment" parameter to anything and add the email you want to the parenthesis (email) and click send

6. Notice that the request is send successfully

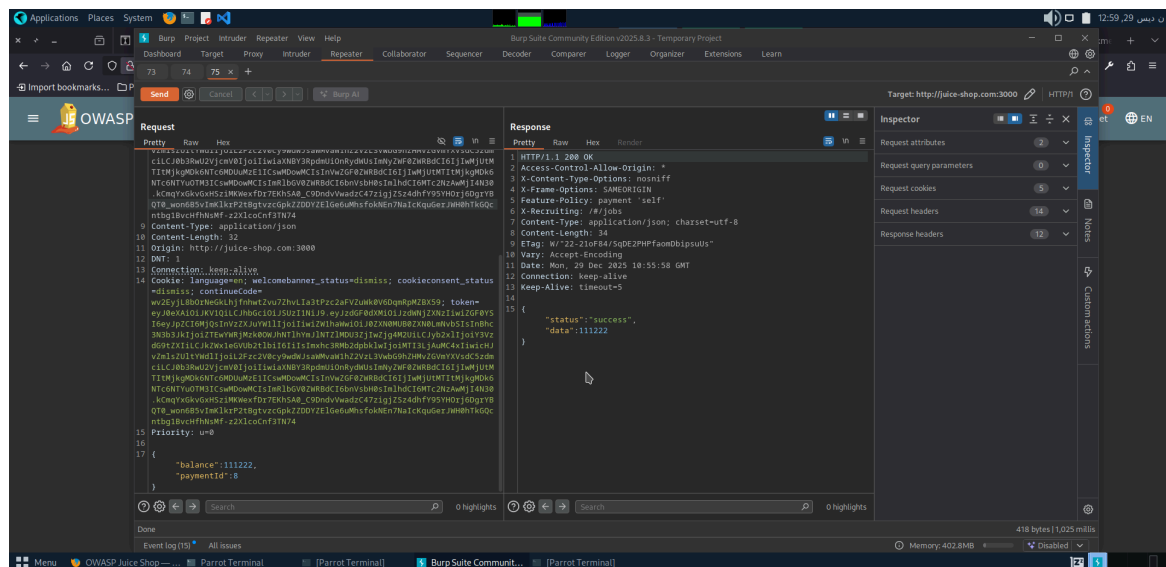
- **Impact:** The potential reputational damage can lead to indirect financial loss to the business, also it's possible to clobber the database with incorrect representation of the data causing a wrong representation of the business performance and consumed time to clean up the inaccurate data.
- **Remediation Advice:** Ensure that server-side validation is implemented for the user input and assume no-trust of the user input.

21. Title: Business logic flaw leads to add more balance to the digital wallet than intended

- **Risk Rating:** CVSS v4.0 Score: 2.3 / Low
- **Summary:** A logic flaw leads to adding more balance to the digital wallet than intended range (1-1000). At best case this is just input limitation bypass which has no security impact, nonetheless it's best practice to ensure consistency between the client-side and server-side validation, However at the worst case this is a direct financial loss that allow user to buy products for free (further test is required in a real environment).
- **Background:** Business logic flaws occur when an application's design or implementation allows an attacker to manipulate the intended workflow of the system.
- **Technical Details & Evidence:**

- **Evidence**

- Lack of backend validation allows to put more balance to wallet than intended



- Payload: "balance":111222
- Endpoint: PUT /rest/wallet/balance
- **Reproducing steps**
 1. Log in with any account
 2. Go to "digital wallet" page
 3. Enter the balance you want (e.g. 50)

4. "Add new card" information and choose it (don't click submit yet)
 5. Open burp suite or any web proxy of choice and make sure that "intercept is on"
 6. Click submit to capture the request
 7. Find the "PUT /rest/wallet/balance" endpoint and right click it and choose "send to repeater"
 8. Navigate to repeater tab and change the "balance" parameter in the body JSON data to any amount you want (e.g. 111222) and click send
 9. A success response should be received and the balance should be in the wallet
- **Impact:** Due to the isolated nature of the testing environment, it could not be verified if this balance manipulation would persist through a real financial settlement process. However, the application's internal state reflects the modified balance as 'Available Funds.' If the application logic relies on this internal balance to authorize purchases, this flaw would lead to **unauthorized procurement of goods (Financial Loss)**.
 - **Remediation Advice:** Ensure that server-side validation is implemented for the user input and assume no-trust of the user input.

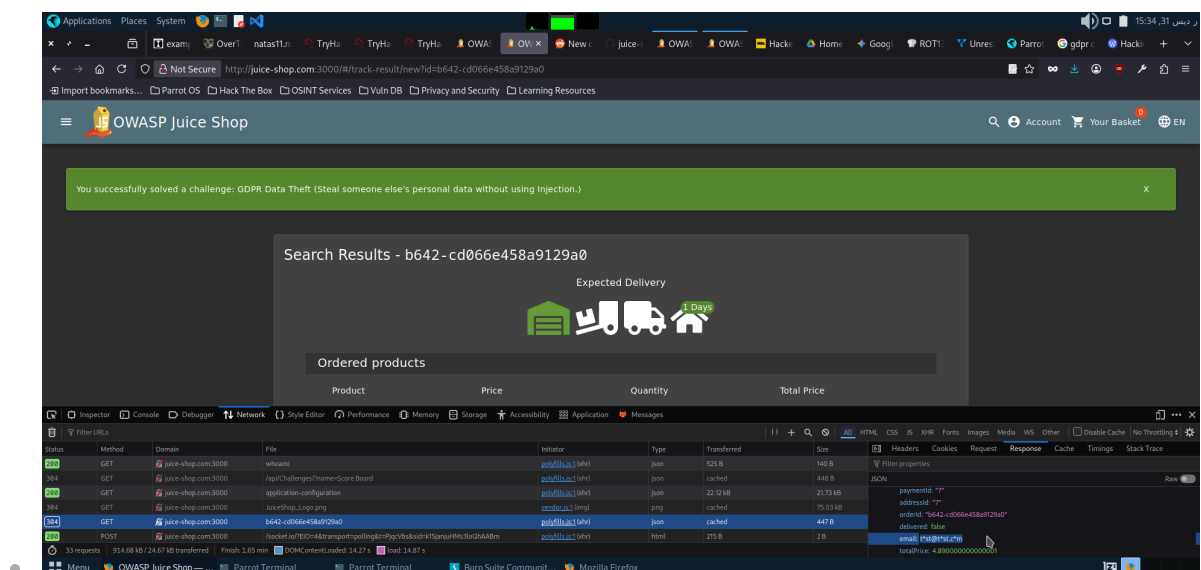
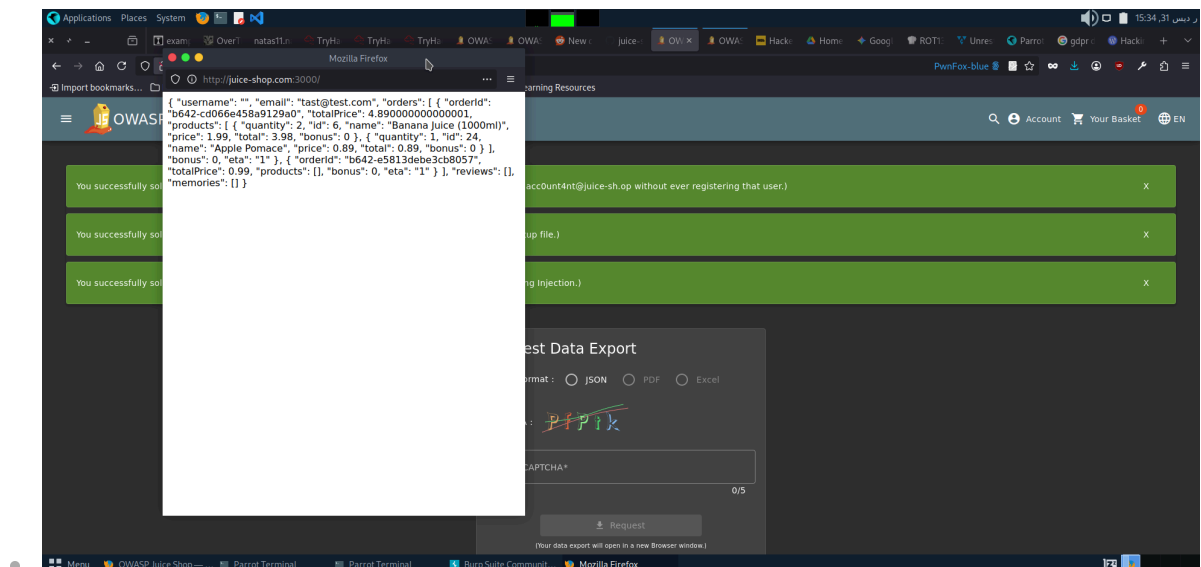
22. **Title:** CSRF in the username change request in the profile page

- **Risk Rating:** CVSS v4.0 Score: 2 / Low
- **Summary:** Due to lack of CSRF-token, it's possible to use a CSRF attack to change the username of a user without their consent from another domain origin. This is a violation of the user's data integrity and can damage the company reputation which can lead to indirect financial loss
- **Background:** A CSRF (cross site request forgery) happens when the web application does not, or cannot, sufficiently verify whether a request was intentionally provided by the user who sent the request, which could have originated from an unauthorized actor.
- **Technical Details & Evidence:**
 - **Evidence**
 - Lack of CSRF-token in the username change request leads to using the CSRF attack to change the username without the consent of the user

23. **Title:** It's possible for one user to access another user's data if the 2 users have a similar email addresses in the request data export functionality

- **Risk Rating:** CVSS v4.0 Score: 2.1 / Low
- **Summary:** When requesting data export if 2 or more emails are similar to each others then it will return data of all emails at the same time. This is an unauthorized access to other users data which violates the confidentiality of their data.
- **Background:** Business logic flaws occur when an application's design or implementation allows an attacker to manipulate the intended workflow of the system.
- **Technical Details & Evidence:**
 - **Evidence**

- Due to improper validation it's possible for one user to access other user's data if the 2 users have a similar (not the same) email address (test@test.com - tast@test.com), this result in the server obfuscating the two emails in the same way which leads to one of those users to be able to retrieve both users data



- **Endpoint:** POST /rest/user/data-export

- **Reproducing steps**

1. Create 2 accounts with a similar emails (e.g. test@test.com - tast@test.com)
2. Login into account 1 (the attacker)
3. Request export data and notice it gives back both accounts data

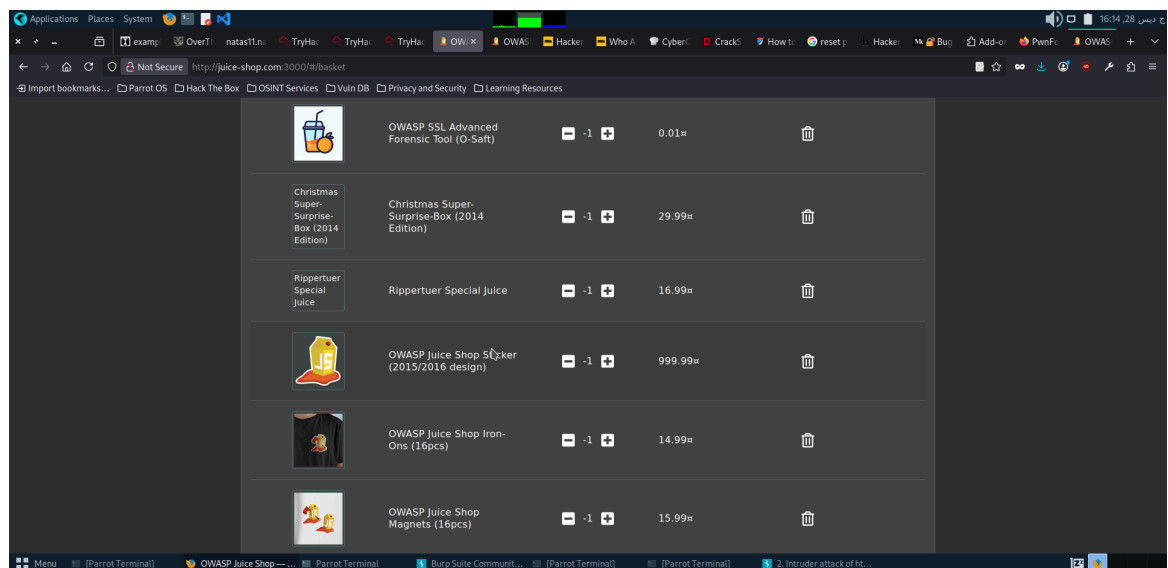
- **Impact:** Unauthorized access to other users data can cause reputational damage to the company and indirect financial loss in the long run
- **Remediation Advice:** Ensure that server-side validation is implemented for the user input and assume no-trust of the user input.

24. **Title:** IDOR leads to unauthorized access to unreleased items

- **Risk Rating:** CVSS v4.0 Score: 2.3 / Low
- **Summary:** An IDOR in the "POST /api/BasketItems/" endpoint can lead to add unreleased items to the basket and purchasing them. Which violates the confidentiality of the company data.
- **Background:** An IDOR vulnerability is when an attacker can gain unauthorized access to other users data through changing the id in the request.
- **Technical Details & Evidence:**

- **Evidence**

- IDOR vulnerability identified through burp intruder leads to unauthorized access to items yet to be released



- Payload: "ProductId": [value to access]
- Endpoint: POST /api/BasketItems/

- **Reproducing steps**

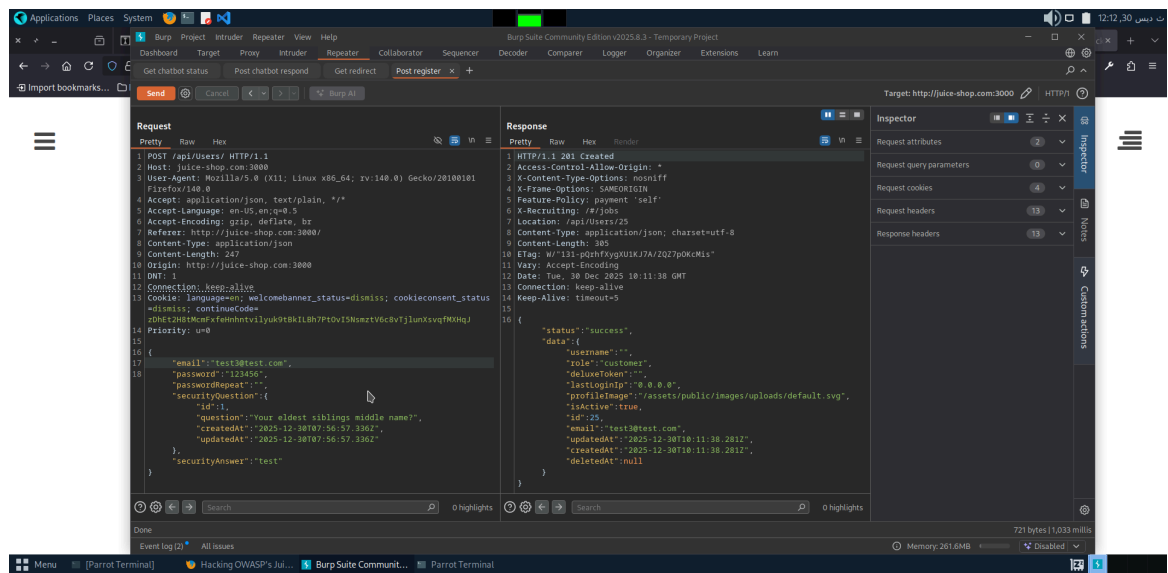
1. Log in with any account
2. Open burp suite or any web proxy of your choice and make sure that "intercept is on"

3. Click "Add to basket" on any product and intercept the request
4. Forward the first "GET" request then right click the "POST /api/BasketItems/" request and choose "send to intruder"
5. Navigate to intruder tab and highlight the value of "ProductId" parameter in the JSON data body and click "Add payload"
6. In the payloads window choose "numbers" from the "payload type" list
7. Set the field "From" to 0 and "To" to 100 and click start attack
8. Wait for it to finish then navigate to the basket and you should see the items is there

- **Impact:** Unauthorized access to unreleased products allows an attacker to bypass business-defined release schedules. This results in the disclosure of proprietary product data, sensitive pricing information, and marketing strategies. Furthermore, if combined with order-manipulation flaws, this could lead to premature inventory depletion and financial discrepancies.
- **Remediation Advice:** Ensure that server-side validation is implemented for the user input and assume no-trust of the user input. Also ensure that a "least privilege policy" is implemented and enforce it the server-side to prevent unauthorized access to others data.

25. **Title:** Registering an account with an empty passwordRepeat field

- **Risk Rating:** CVSS v4.0 Score: 0 / None
- **Summary:** A logic flaw exist in the registration functionality that allows the user to send the request with empty or wrong "passwordRepeat" parameter. It doesn't carry out any security threat, nonetheless, it's best practice to ensure consistency between client-side and server-side validation.
- **Background:** Business logic flaws occur when an application's design or implementation allows an attacker to manipulate the intended workflow of the system.
- **Technical Details & Evidence:**
 - **Evidence**
 - Lack of server side validation leads to register a user with an empty passwordReapeat field

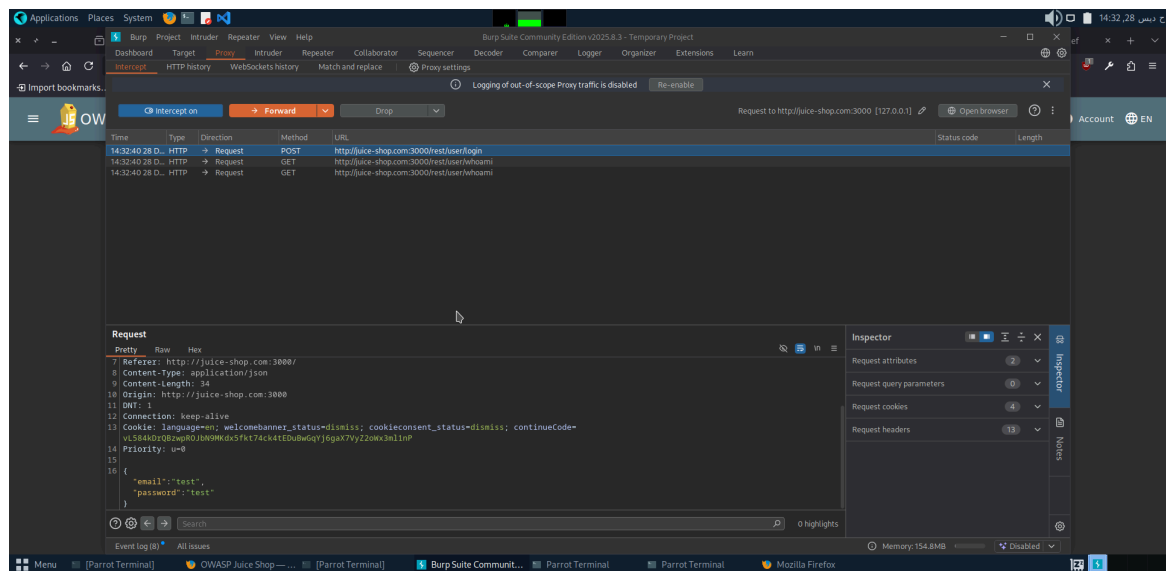


- Endpoint: POST /api/Users/
- **Reproducing steps**
 1. Go to registration page
 2. Open burp suite or any web proxy of your choice and make sure that "intercept is on"
 3. Fill in the registration form and click "Register" to intercept the request
 4. Change the "passwordRepeat" parameter in JSON data body to empty string "" and click send
 5. You should see success in the response
- **Impact:** It carries no security impact but the best practice is to ensure consistency across client-side and server-side validation to ensure consistency across the data in the database
- **Remediation Advice:** Ensure that server-side validation is implemented for the user input and assume no-trust of the user input.

2.2 A04:2025 - Cryptographic Failures

26. Title: Clear text communication over HTTP

- **Risk Rating:** CVSS v4.0 Score: 7.6 / High
- **Summary:** The entire website is transmitted over insecure protocol "HTTP" and not "HTTPS" which can lead to the data being sniffed by unauthorized actor (the attacker).
- **Background:** Cleartext transmission of sensitive information happens when product transmits sensitive or security-critical data in cleartext in a communication channel that can be sniffed by unauthorized actors.
- **Technical Details & Evidence:**
 - **Evidence**
 - Clear text communication over HTTP and not HTTPS



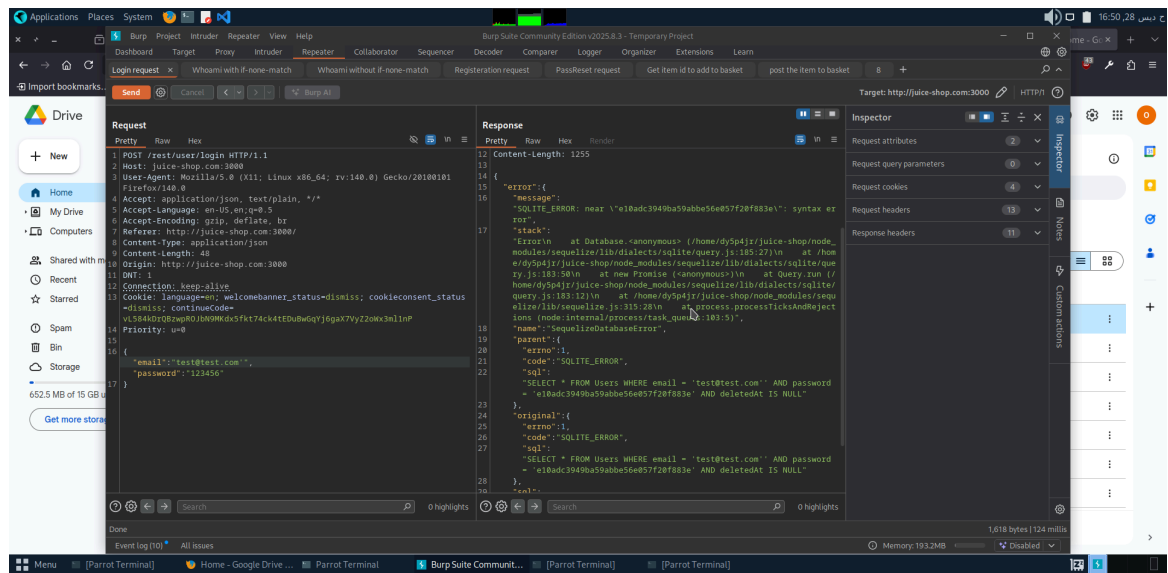
- **Reproducing steps**

1. Go to login page and fill the form (but don't press log in yet)
2. Open burp suite or any web proxy of your choice and make sure that "intercept is on"
3. Press "Login"
4. See the password transmitted in cleartext in burp

- **Impact:** Data transmitted in cleartext communication are susceptible to sniffing attacks from an unauthorized actor, which can leak sensitive data like passwords or PII (personal identifiable information) user data
- **Remediation Advice:** Before transmitting, encrypt the data using reliable, confidentiality-protecting cryptographic protocols like TLS.

27. **Title:** Weak hashing algorithm (md5) for passwords

- **Risk Rating:** CVSS v4.0 Score: 5.3 / Medium
- **Summary:** Weak hashing algorithms like (md5) are not sufficient to protect the passwords, if any breach happens and a threat actor get ahold of the database, he then can easily crack all the passwords in minutes. This can cause a massive ATO (account take over) or other accounts on other websites being breached if the same password is in use
- **Background:** Obscuring a password with a trivial encoding that does not protect the password.
- **Technical Details & Evidence:**
 - **Evidence**
 - Weak hashing algorithm (md5) for passwords (hash exposed via SQL verbose error message in the login form)



• Reproducing steps

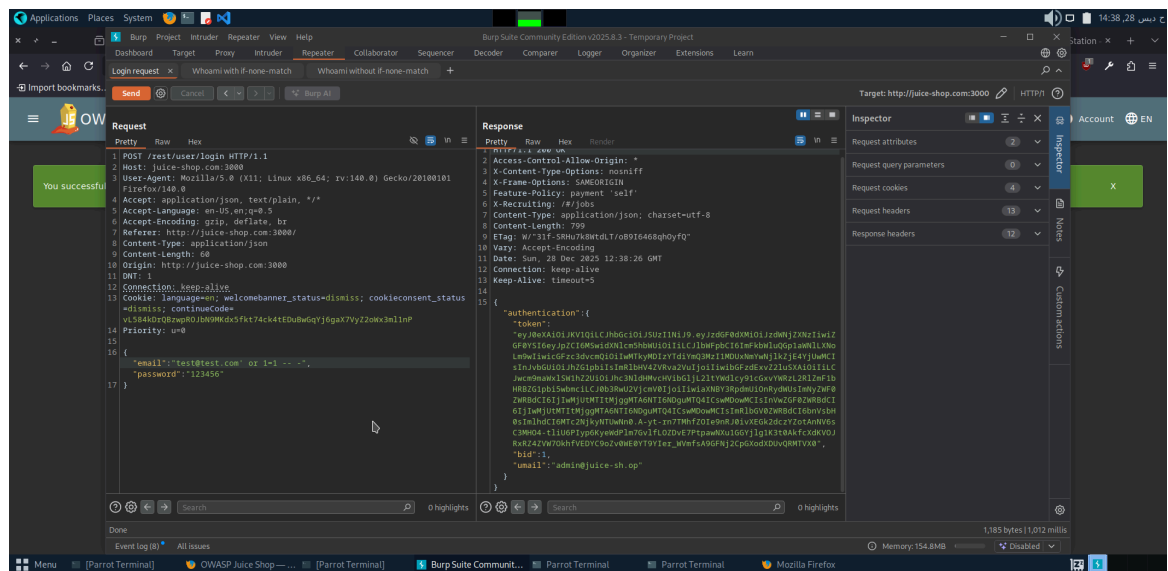
1. Open the log in page
2. Open burp suite or any other web proxy of your choice and make sure that "intercept is on"
3. fill in the login form and click "Login" to intercept the request
4. Right click the login request "POST /rest/user/login" and inject a single quote (') in the "email" parameter in the JSON data body and click send
5. Notice the password is hashed using md5 in the response

- **Impact:** A threat actor getting ahold of the database can crack all the passwords in minutes, and can cause massive ATO (account take over) on the website, not only that but if users using the same passwords on other websites it can result in accounts breached in other platforms as well
- **Remediation Advice:** Passwords should be encrypted with keys that are at least 128 bits in length for adequate security.

2.3 A05:2025 - Injection

28. **Title:** SQLi via email field of the login page

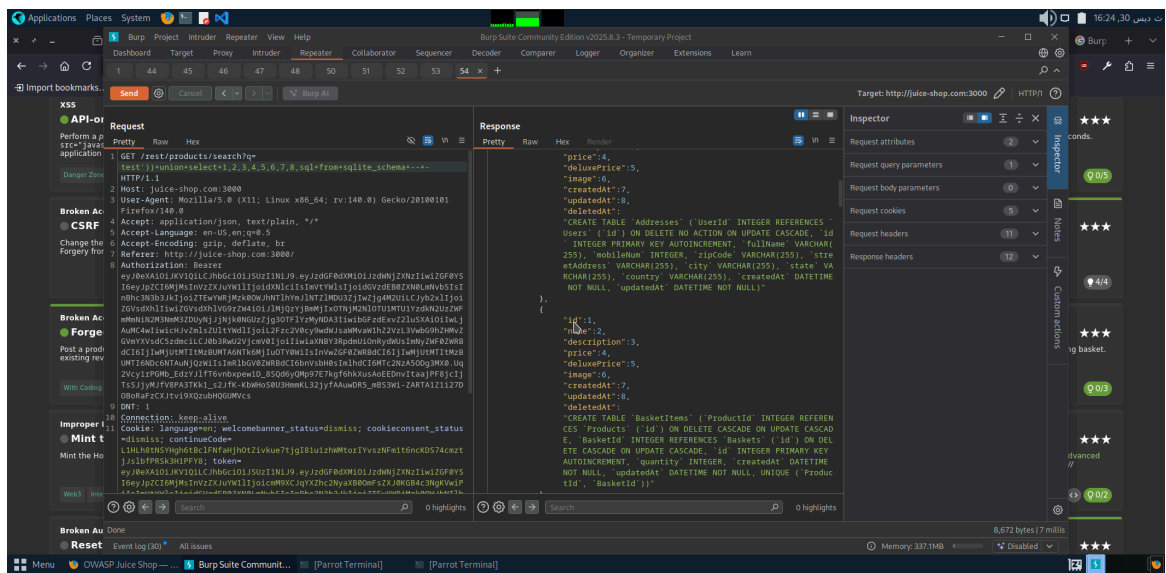
- **Risk Rating:** CVSS v4.0 Score: 9.3 / Critical
- **Summary:** Using SQLi (SQL injection) it's possible to bypass authentication process in the login page and log in with any account.
- **Background:** SQLi (SQL injection) is when application trust user input and concatenate the input directly in the query statement.
- **Technical Details & Evidence:**
 - **Evidence**
 - SQLi via email field of the login page leads to unauthorized access to the admin account



- Payload: test@test.com or 1=1 -- -
- Endpoint: POST /rest/user/login
- **Reproducing steps**
 1. Open the log in page
 2. Enter the following payload in the email field (' or 1=1 -- -) (alternatively you can enter the email you want to access and comment the password e.g. test@test.com -- -)
 3. You should successfully log in to the first account returned from the database (or the account you chose to access)
- **Impact:** SQLi is dangerous due to the ability to inject SQL code directly and cause unauthenticated access or access confidential data stored in the database.
- **Remediation Advice:** Usage of parametrized queries mitigate the SQLi threat.

29. Title: SQLi in the search API

- **Risk Rating:** CVSS v4.0 Score: 7.6 / High
- **Summary:** Using SQLi (SQL injection) it's possible to retrieve database information.
- **Background:** SQLi (SQL injection) is when application trust user input and concatenate the input directly in the query statement.
- **Technical Details & Evidence:**
 - **Evidence**
 - SQLi identified through manual testing in search api endpoint leads to exfiltrate database information



- Payload: test'))+union+select+1,2,3,4,5,6,7,8,sql+from+sqlite_schema+---+-
- Endpoint: GET /rest/products/search?q=

• Reproducing steps

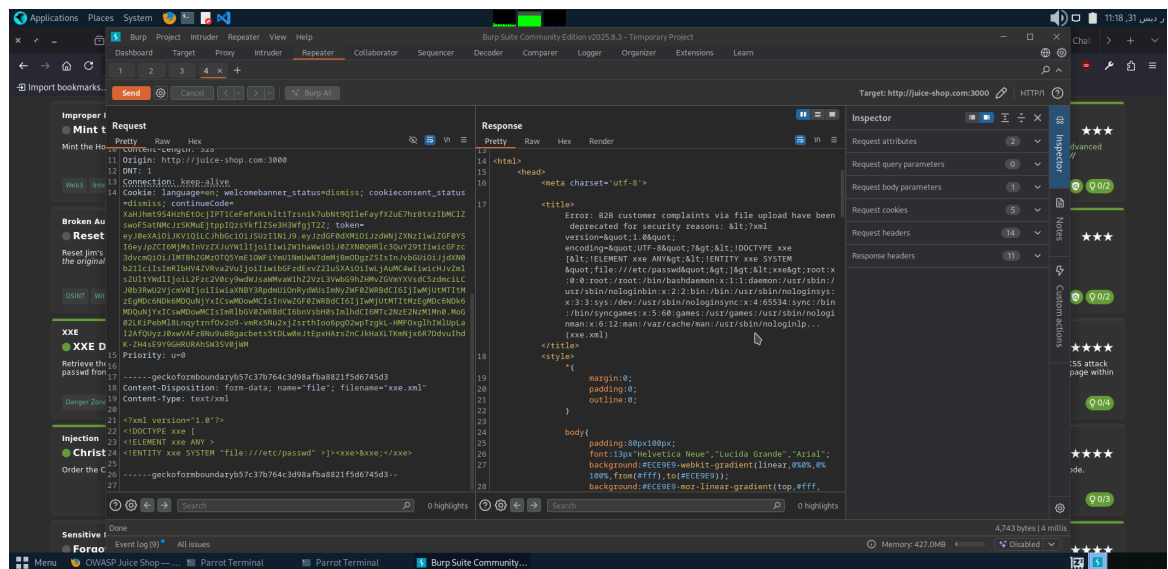
1. Log in with any account
2. Open burp suite or any preferred proxy and make sure "intercept is on"
3. Refresh the page to capture the request
4. Find "GET /rest/products/search?q=" endpoint
5. Right click it and choose "send to repeater"
6. Navigate the repeater tab and type the following payload in the "q" parameter in the endpoint (test'))+union+select+1,2,3,4,5,6,7,8,sql+from+sqlite_schema+---+- and click send
7. You should receive the entire database schema in the response

- **Impact:** SQLi is dangerous due to the ability to inject SQL code directly and cause unauthenticated access or access confidential data stored in the database.
- **Remediation Advice:** Usage of parametrized queries mitigate the SQLi threat.

30. Title: XXE via file upload in the complain form

- **Risk Rating:** CVSS v4.0 Score: 7.1 / High
- **Summary:** XXE (XML external entity) injection is possible through file upload in the "complaints" page leading to access arbitrary files on the system.
- **Background:** Improper restriction of XML external entity reference is when application processes an XML document that can contain XML entities with URIs that resolve to documents outside of the intended sphere of control, causing the product to embed incorrect documents into its output.
- **Technical Details & Evidence:**
 - **Evidence**

- Due to improper server side validation and a usage of a unsuccessfully deprecated B2B invoice complaint form it's possible to unauthorized access to system files using an XXE vulnerability



- Payload: `<?xml version="1.0"?>`
`<!DOCTYPE xxe [`
`<!ELEMENT xxe ANY >`
`<!ENTITY xxe SYSTEM "file:///etc/passwd" >]><xxe>&xxe;</xxe>`

- Endpoint: POST /file-upload

• Reproducing steps

1. Log in with any account
2. Navigate to "complaints" page
3. Make a new XML file on your system and add the following payload as the content of the file `<?xml version="1.0"?>`
`<!DOCTYPE xxe [`
`<!ELEMENT xxe ANY >`
`<!ENTITY xxe SYSTEM "file:///etc/passwd" >]><xxe>&xxe;</xxe>`
4. Open burp suite or any preferred web proxy and make sure "intercept is on"
5. Fill in the form and upload the XML file then click submit to intercept the request
6. Find "POST /file-upload" , right click it and choose "send to repeater"
7. Navigate to repeater tab and and click "Send"
8. You should receive the content of "/etc/passwd" in the response

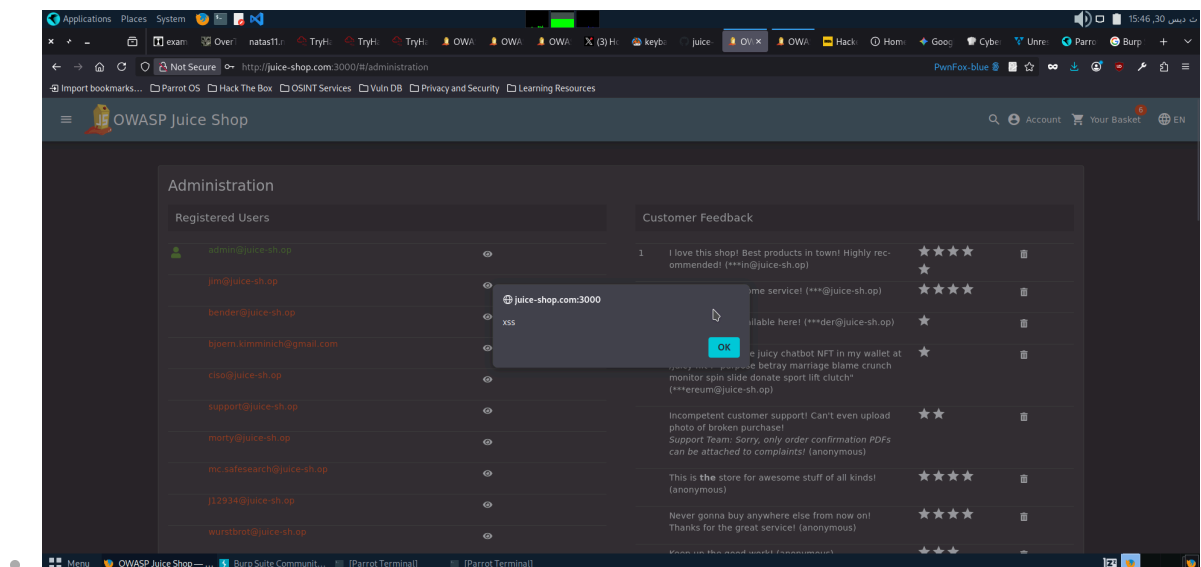
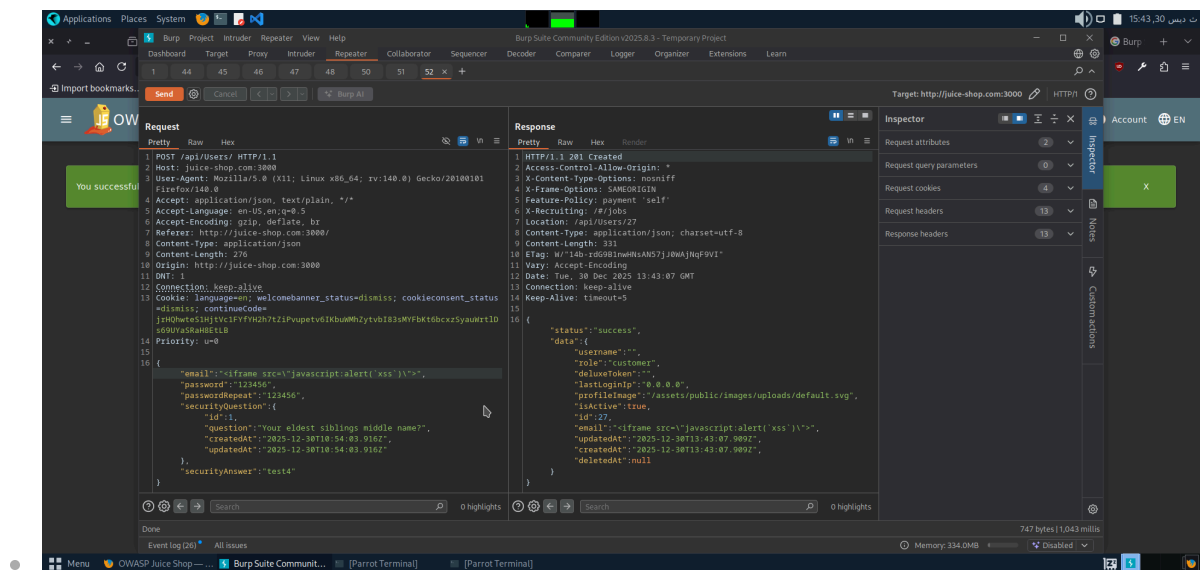
- **Impact:** If the attacker is able to include a crafted DTD and a default entity resolver is enabled, the attacker may be able to access arbitrary files on the system.
- **Remediation Advice:** Many XML parsers and validators can be configured to disable external entity expansion.

31. **Title:** Stored XSS in the email field of registration form

- **Risk Rating:** CVSS v4.0 Score: 6.9 / Medium
- **Summary:** XSS (cross site scripting) in the "email" field of registration page fire up in the administration panel. It can lead to stealing admin session cookies
- **Background:** The web application receives input from an upstream component, but it does not neutralize or incorrectly neutralizes special characters such as "<", ">", and "&" that could be interpreted as web-scripting elements when they are sent to a downstream component that processes web pages.
- **Technical Details & Evidence:**

- **Evidence**

- XSS (cross site scripting) is identified in the email field of registration by bypassing client side protection using a proxy which leads to the payload being fired up in the administration panel



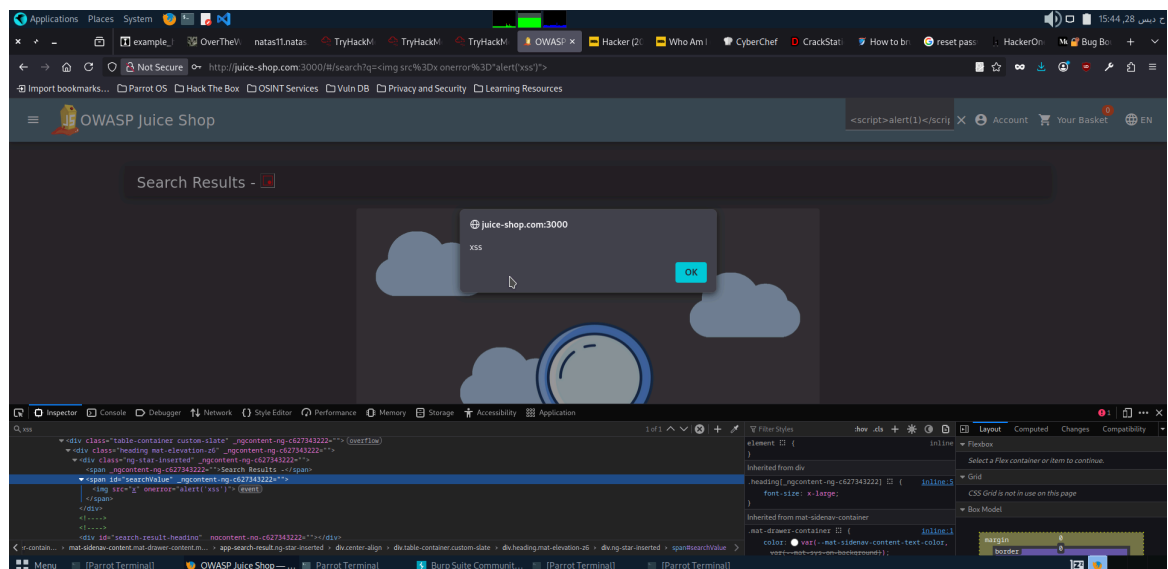
- Payload: `<iframe src="javascript:alert('xss')">`
- Endpoint: POST /api/Users/
- **Reproducing steps**

1. Go to registration page
2. Open burp suite or any web proxy of your choice and make sure "intercept is on"
3. Fill in the registration form and click "register" to intercept the request
4. Find "POST /api/Users/" navigate to JSON data body and change the value of "email" parameter to "`<iframe src='javascript:alert( xss )'>`" and forward the request
5. Go to administration panel with admin account and the XSS should trigger

- **Impact:** XSS (cross site scripting) can be used to steal user's session cookies or run arbitrary JavaScript code in their browser.
- **Remediation Advice:** Encode all user-provided data before rendering it to the page. This converts special characters (like `<` to `<`;) so the browser treats them as text, not executable code.

32. Title: DOM XSS in the search functionality

- **Risk Rating:** CVSS v4.0 Score: 5.3 / Medium
- **Summary:** DOM XSS (cross site scripting) in the main search functionality of the web application can lead to stealing users session cookies and running arbitrary JavaScript code in their browser
- **Background:** The web application receives input from an upstream component, but it does not neutralize or incorrectly neutralizes special characters such as "`<`", "`>`", and "`&`" that could be interpreted as web-scripting elements when they are sent to a downstream component that processes web pages.
- **Technical Details & Evidence:**
 - **Evidence**
 - Identified DOM XSS in the search functionality due to lack of input sanitization



- **Payload:** `<img src=x onerror="alert('xss')">`

- Endpoint: <http://juice-shop.com:3000/#/search?q=>

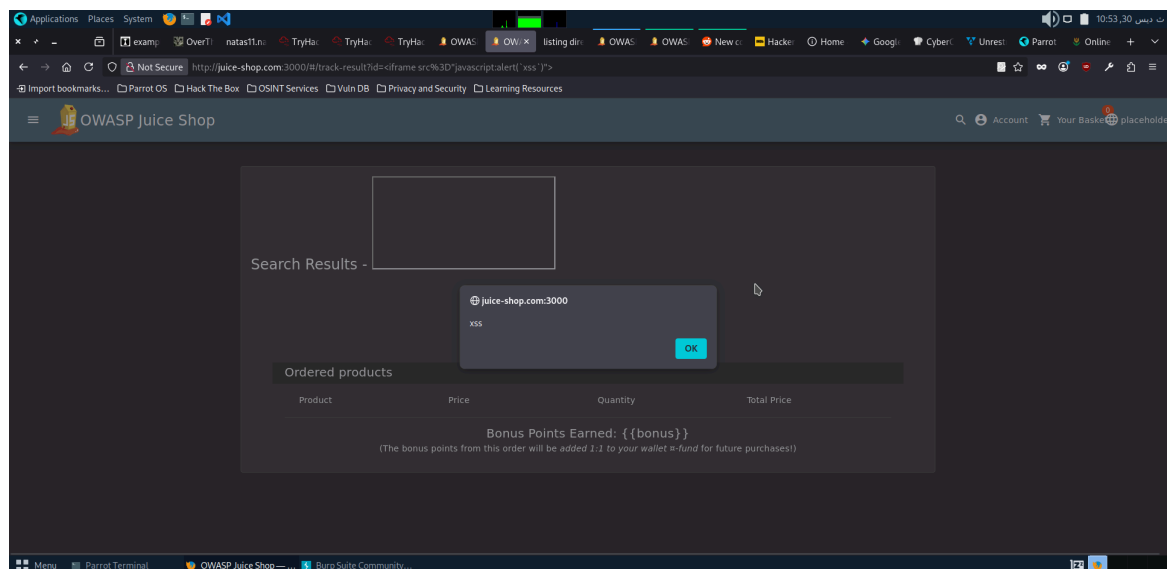
- **Reproducing steps**

1. Log in with any account
2. In the search bar type the following payload "`<iframe src='javascript:alert( xss )'>`" and press enter
3. XSS should trigger in your browser with popup

- **Impact:** XSS (cross site scripting) can be used to steal user's session cookies or run arbitrary JavaScript code in their browser.
- **Remediation Advice:** Encode all user-provided data before rendering it to the page. This converts special characters (like `<` to `<`;) so the browser treats them as text, not executable code.

33. Title: Reflected XSS in the URL parameter in the track order page

- **Risk Rating:** CVSS v4.0 Score: 4.8 / Medium
- **Summary:** Reflected XSS (cross site scripting) in the URL parameter in the track order page can lead to stealing users session cookies and running arbitrary JavaScript code in their browser.
- **Background:** The web application receives input from an upstream component, but it does not neutralize or incorrectly neutralizes special characters such as "<", ">", and "&" that could be interpreted as web-scripting elements when they are sent to a downstream component that processes web pages.
- **Technical Details & Evidence:**
 - **Evidence**
 - Lack of input sanitization in the id URL parameter in track order page leads to a reflected XSS vulnerability



- Payload: `track-result?id=<iframe src='javascript:alert( xss )'>`

- Endpoint: <http://juice-shop.com:3000/#/track-result?id=>

- **Reproducing steps**

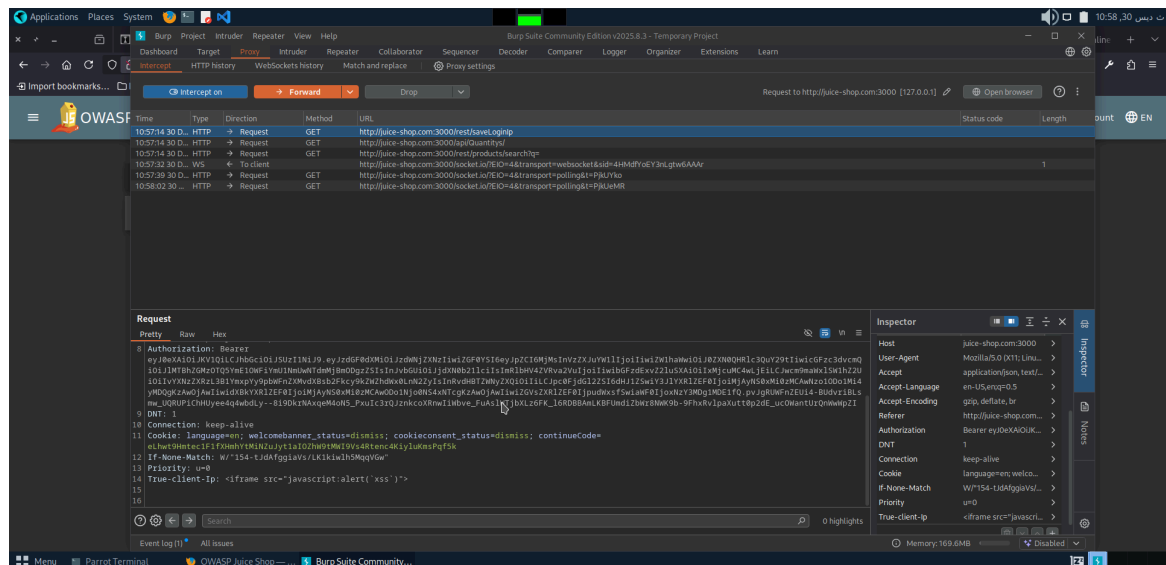
1. Log in with any account
2. Make a purchase and navigate to "track order" page
3. Inject the following payload "`<iframe src='javascript:alert( xss )'>`" in the URL parameter `"?id="` and press enter
4. XSS should trigger with a popup

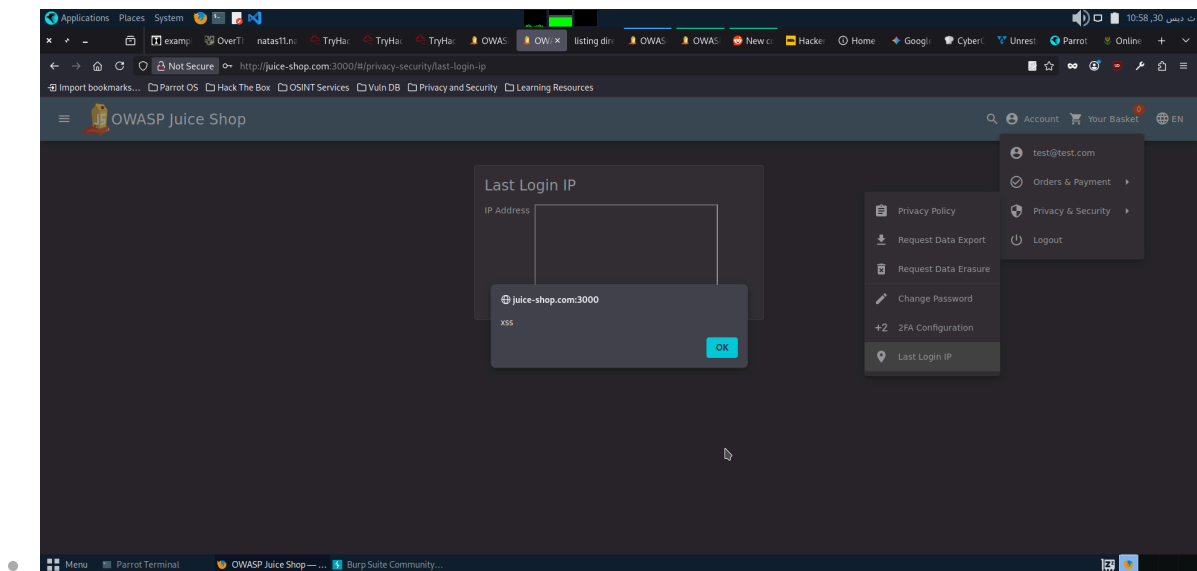
- **Impact:** XSS (cross site scripting) can be used to steal user's session cookies or run arbitrary JavaScript code in their browser.
- **Remediation Advice:** Encode all user-provided data before rendering it to the page. This converts special characters (like `<` to `<`;) so the browser treats them as text, not executable code.

34. **Title:** HTTP-Header stored XSS in the `"/saveLoginIp"` endpoint through the `"True-Client-IP"` header

- **Risk Rating:** CVSS v4.0 Score: 4.8 / Medium
- **Summary:** HTTP-Header stored XSS (cross site scripting) in the `"/saveLoginIp"` endpoint through the `"True-Client-IP"` header can lead to stealing users session cookies and running arbitrary JavaScript code in their browser.
- **Background:** The web application receives input from an upstream component, but it does not neutralize or incorrectly neutralizes special characters such as `"<"`, `">"`, and `"&"` that could be interpreted as web-scripting elements when they are sent to a downstream component that processes web pages.
- **Technical Details & Evidence:**
 - **Evidence**

- Lack of server side validation leads to xss in `"True-Client-IP HTTP-Header"` which get triggered after visiting `"last login ip"` page





- Payload: `True-Client-IP:<iframe src="javascript:alert( xss )">`
- Endpoints: GET `/rest/saveLoginIp` , <http://juice-shop.com:3000/#/privacy-security/last-login-ip>

- **Reproducing steps**

1. Log in with any account
2. Open burp suite or any proxy of your choice and make sure "intercept is on"
3. Log out and intercept the request
4. Find "GET `/rest/saveLoginIp`" endpoint and add the "True-Client-IP" header with the following payload "`<iframe src="javascript:alert('xss')">`" and forward the request
5. Log in into the same account again and navigate to the "Last login ip" page
6. XSS should trigger with a popup

- **Impact:** XSS (cross site scripting) can be used to steal user's session cookies or run arbitrary JavaScript code in their browser.
- **Remediation Advice:** Encode all user-provided data before rendering it to the page. This converts special characters (like `<` to `<` ;) so the browser treats them as text, not executable code.

35. **Title:** Stored XSS in the username field in the profile page

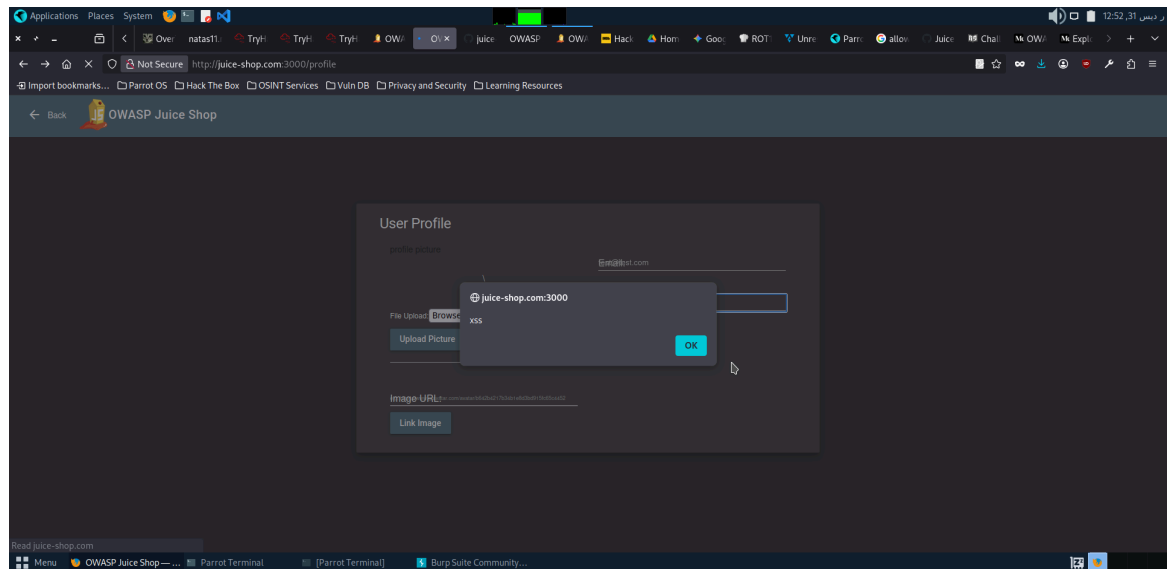
- **Risk Rating:** CVSS v4.0 Score: 4.8 / Medium
- **Summary:** Stored XSS (cross site scripting) in the username field in the "profile" page is possible through bypassing CSP (content security policy) through "imageUrl" field and bypass the encoding the "username" field can lead to stealing users session cookies and running arbitrary code in their browser.
- **Background:** The web application receives input from an upstream component, but it does not neutralize or incorrectly neutralizes special characters such as `"<"`, `">"`, and `"&"` that

could be interpreted as web-scripting elements when they are sent to a downstream component that processes web pages.

- **Technical Details & Evidence:**

- **Evidence**

- XSS (cross site scripting) is identified in the username field due to improper sanitization, also it's possible to bypass the CSP of the website through injecting a semicolon ";" and use a bypass payload to CSP to "unsafe-inline"



- Payloads: `<<a>ascript>alert('xss')</script>`
; script-src-elem 'self' 'unsafe-inline' 'unsafe-eval'
 - Endpoint: POST /profile

- **Reproducing steps**

1. Log in with any account
2. Navigate to the profile page
3. Inject the following payload in the "username" field `<<a>ascript>alert('xss')</script>`
4. Inject the following payload in the "image URL" field (; script-src-elem 'self' 'unsafe-inline' 'unsafe-eval')
5. Refresh the page and XSS should trigger with a popup

- **Impact:** XSS (cross site scripting) can be used to steal user's session cookies or run arbitrary JavaScript code in their browser.
- **Remediation Advice:** Encode all user-provided data before rendering it to the page. This converts special characters (like < to < ;) so the browser treats them as text, not executable code.

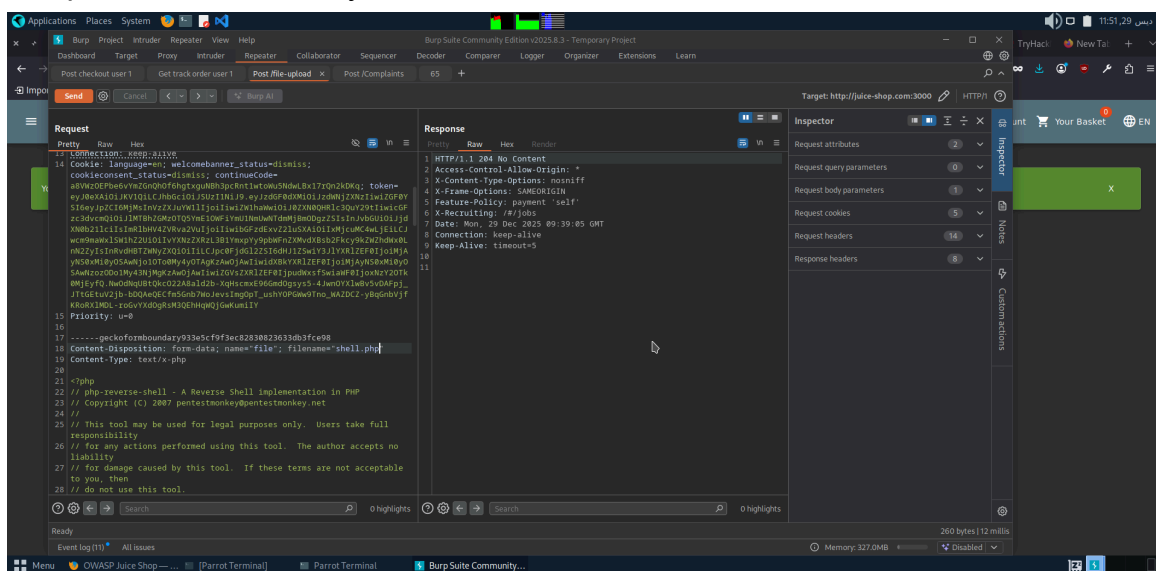
2.4 A06:2025 - Insecure Design

36. **Title:** Uploading any file type via the complaints form

- **Risk Rating:** CVSS v4.0 Score: 2.3 / Low
- **Summary:** It's possible to bypass file upload mechanism in the "complaints" page by renaming the file with an allowed extension (e.g. shell.png) and changing the header of the file then intercept the request and change the name and type (e.g. shell.php). Worst case scenario this can lead to remote code execution on the server, but due to failure of executing the file the impact is lowered.
- **Background:** Unrestricted upload of file and dangerous type is when the application allows the upload or transfer of dangerous file types that are automatically processed within its environment.
- **Technical Details & Evidence:**

- **Evidence**

- Lack of backend validation and trust client side validation only results in successfully uploading php files leading to upload a webshell that can compromise the entire system to unauthorized access

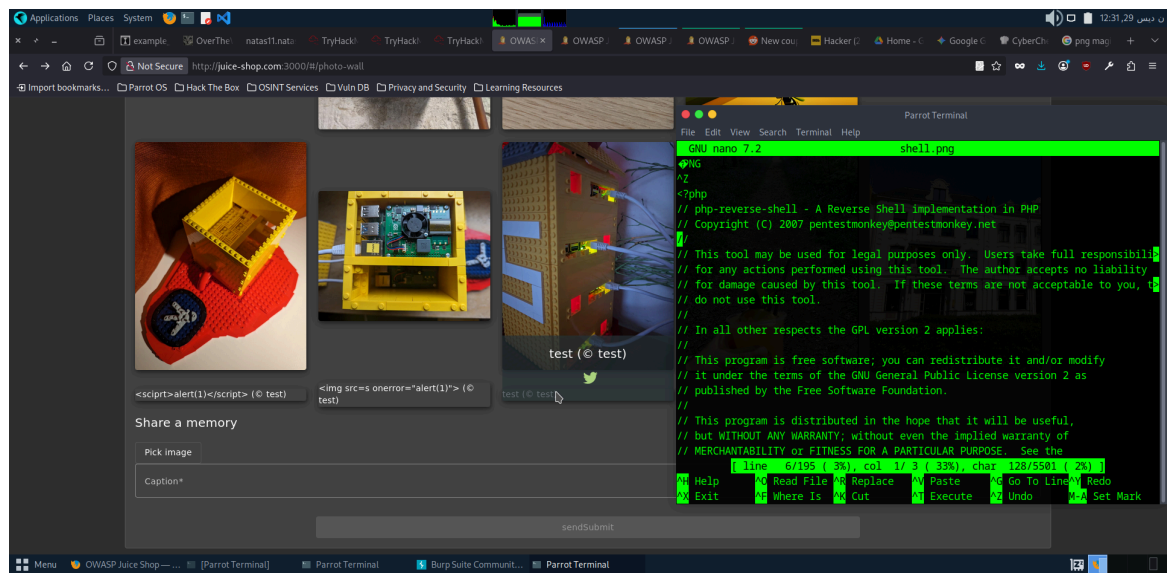


- Payload: filename="shell.php" - Content-Type: text/x-php
- Endpoint: POST /file-upload
- **Reproducing steps**
 1. Download this crafted shell "<https://github.com/newtonys/Exploit-Scripts/blob/main/shell.pdf>" and change the "ip" and "port" in the script to yours (skip the next steps of creating your own shell)
 2. If you'd prefer to create your own shell then follow these steps
 1. Download a php reverse shell or a webshell from pentestmonkey "<https://pentestmonkey.net/tag/php>"
 2. Open the file in a hexeditor (e.g. hexedit)
 3. Change the first 5 bytes to "25 50 44 46 2D" (these known as pdf magic bytes)
 4. Save the changes and exit

5. Rename the file to "[your preferred name].pdf"
 3. Log in with any account
 4. Navigate to the "complaints" page
 5. Open burp suite or any preferred web proxy and make sure that "intercept is on"
 6. Fill in the form and choose your file (e.g. shell.pdf) and submit to intercept the request
 7. In the request, change the "filename" to "shell.php" and the "Content-Type" to "text/x-php"
 8. Forward the request and the shell should be uploaded
- **Impact:** The impact of this vulnerability is usually high or critical but due to failure of accessing the file and executing it to gain RCE (remote code execution) the impact is set low, nonetheless even with failure of executing the file it's best practice to restrict the file upload to the correct types to avoid any devastating outcome in the future.
 - **Remediation Advice:** Consider storing the uploaded files outside of the web document root entirely. Then, use other mechanisms to deliver the files dynamically.

37. **Title:** Uploading any file type via photo wall form

- **Risk Rating:** CVSS v4.0 Score: 2.3 / Low
- **Summary:** It's possible to bypass file upload mechanism in the "photo wall" page by renaming the file with an allowed extension (e.g. shell.png) and changing the header of the file then intercept the request and change the name and type (e.g. shell.php). Worst case scenario this can lead to remote code execution on the server, but due to failure of executing the file the impact is lowered.
- **Background:** Unrestricted upload of file and dangerous type is when the application allows the upload or transfer of dangerous file types that are automatically processed within its environment.
- **Technical Details & Evidence:**
 - **Evidence**
 - Lack of file content validation leads to bypass the restriction of uploading only image and can upload other file's types



- Payload:
- Endpoint: POST /rest/memories

- **Reproducing steps**

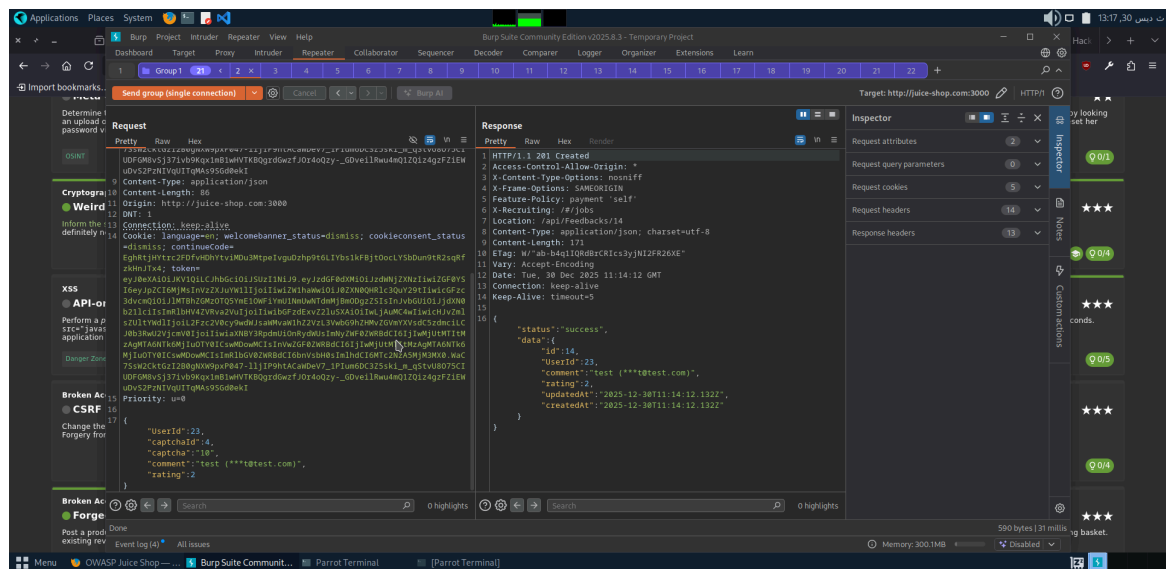
1. Download this crafted shell "<https://github.com/newtonys/Exploit-Scripts/blob/main/shell.png>" and change the "ip" and "port" in the script to yours (skip the next steps of creating your own shell)
2. If you'd prefer to create your own shell then follow these steps
 1. Download a php reverse shell or a webshell from pentestmonkey "<https://pentestmonkey.net/tag/php>"
 2. Open the file in a hexeditor (e.g. hexedit)
 3. Change the first 8 bytes to " 89 50 4E 47 0D 0A 1A 0A " (these known as png magic bytes)
 4. Save the changes and exit
 5. Rename the file to "[your preferred name].png"
3. Log in with any account
4. Navigate to the "photo wall" page
5. Open burp suite or any preferred web proxy and make sure that "intercept is on"
6. Fill in the form and choose your file (e.g. shell.png) and submit to intercept the request
7. In the request, change the "filename" to "shell.php" and the "Content-Type" to "text/x-php"
8. Forward the request and the shell should be uploaded

- **Impact:** The impact of this vulnerability is usually high or critical but due to failure of accessing the file and executing it to gain RCE (remote code execution) the impact is set low, nonetheless even with failure of executing the file it's best practice to restrict the file upload to the correct types to avoid any devastating outcome in the future.

- **Remediation Advice:** Consider storing the uploaded files outside of the web document root entirely. Then, use other mechanisms to deliver the files dynamically.

38. Title: CAPTCHA bypass in the feedback functionality

- **Risk Rating:** CVSS v4.0 Score: 2.3 / Low
- **Summary:** CAPTCHA bypass is possible due to validation of the "captchaId" for multiple usage. This render the CAPTCHA security mechanism unreliable and can clobber the server with 1000s of fake feedbacks.
- **Background:** Reliance on untrusted inputs in a security decision is when application uses a protection mechanism that relies on the existence or values of an input, but the input can be modified by an untrusted actor in a way that bypasses the protection mechanism.
- **Technical Details & Evidence:**
 - **Evidence**
 - CAPTCHA can be bypassed due to improper implementation of the CAPTCHA check in the server side



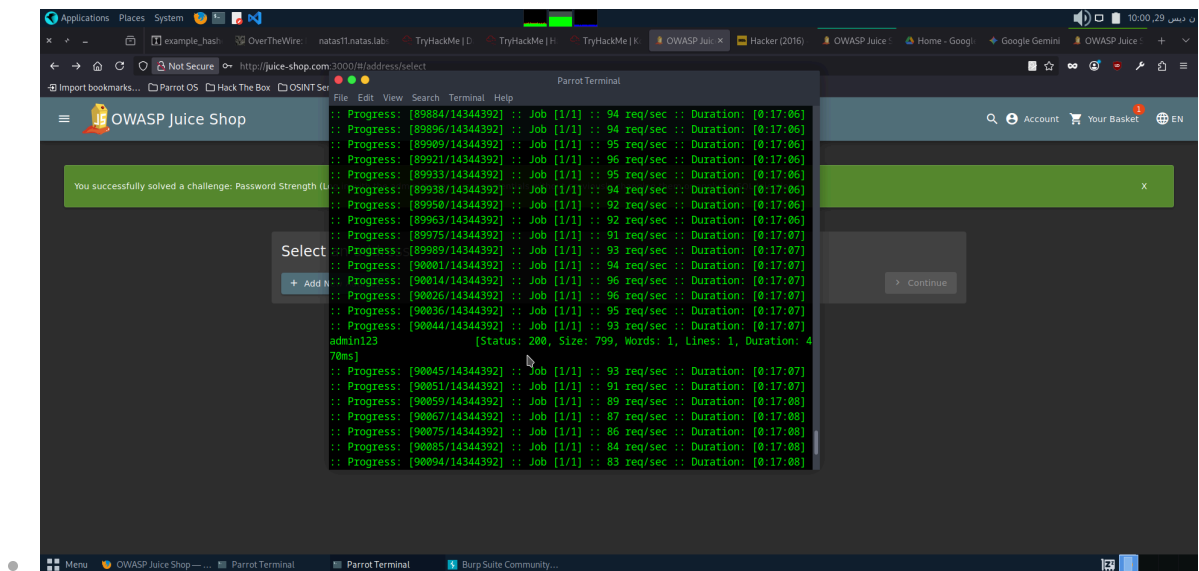
- Endpoint: POST /api/Feedbacks/
- **Reproducing steps**
 1. Log in with any account
 2. Navigate to "feedback" page
 3. Open burp suite or any other web proxy of your choice and make sure that "intercept is on"
 4. Fill in the "feedback" form with any data and submit to capture the request
 5. Find "POST /api/Feedbacks/" endpoint and right click it and choose "send to repeater"
 6. Navigate to the repeater tab and click the add tab button "+" and select "Create tab group to display the create group context menu"

7. Enter a "Group name" and click "Select all" to add all tabs to the group, or use the checkboxes to pick specific ones (you can shift-click to select multiple tabs)
 8. Select a group color (the group's tabs are highlighted with this color on the tab bar)
 9. Click "Create" to set up the group (an icon for the group is added to the tab bar. click the icon to display the individual tabs within the group)
 10. Right click the request tab and choose "Duplicate" then enter the number (e.g. 20)
 11. From the down arrow next to "Send" button choose "Send Group (single connection)" and click it
 12. You should receive a success in the response
- **Impact:** The ability to bypass CAPTCHA and clobber the server of 1000s of fake feedbacks can cause time loss to handle feedbacks and cleaning the database of incorrect data.
 - **Remediation Advice:** Ensure that the system definitively and unambiguously keeps track of its own state and user state and has rules defined for legitimate state transitions. Do not allow any application user to affect state directly in any way other than through legitimate actions leading to state transitions.

2.5 A07:2025 - Authentication Failures

39. **Title:** Brute force users accounts passwords in the login form

- **Risk Rating:** CVSS v4.0 Score: 8.6 / High
- **Summary:** Due to lack of rate limiting it's possible to use automated tools or scripts to brute force the passwords in the login form. This result in ATO (account take over) and unauthorized access to users accounts
- **Background:** Improper restriction of excessive authentication attempts is when application does not implement sufficient measures to prevent multiple failed authentication attempts within a short time frame.
- **Technical Details & Evidence:**
 - **Evidence**
 - Lack of rate limiting leading to brute force other users accounts passwords



- Payload: `ffuf -request request.txt -request-proto http -w /usr/share/wordlists/rockyou.txt -mc 200`
- Endpoint POST `/rest/user/login`

- **Reproducing steps**

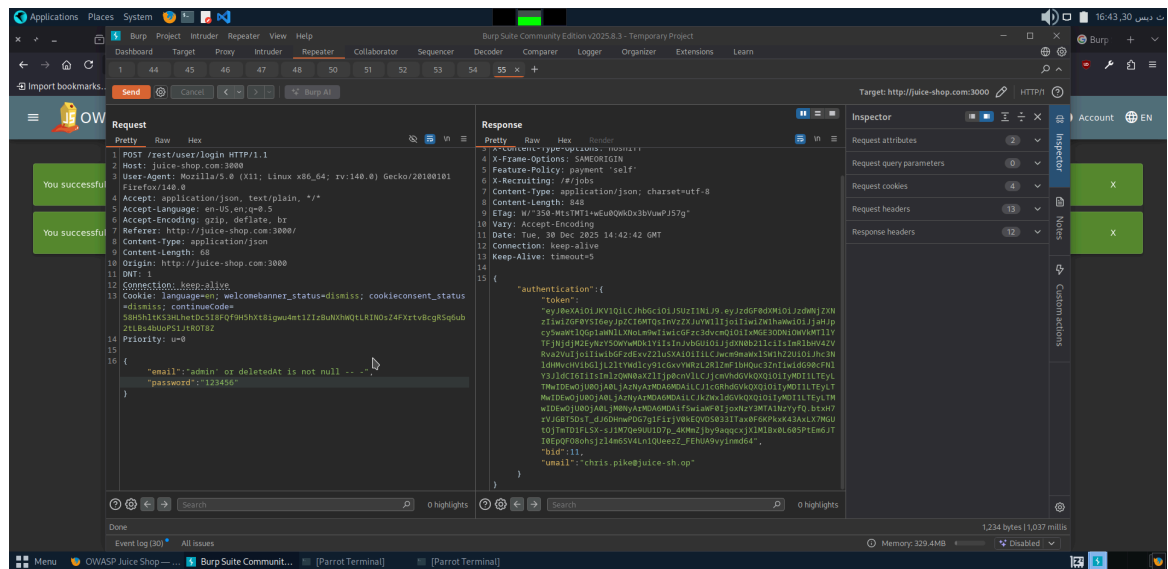
1. Open the log in page
2. Open burp suite or any proxy of your choice and make sure "intercept is on"
3. Fill in the log in form with a valid account and any password and click "Login" to intercept the request
4. Find "POST `/rest/user/login`" endpoint and right click it and choose "send to repeater"
5. Navigate to repeater tab and change the "password" parameter's value to "FUZZ" in the JSON data body
6. Copy the entire request and paste it into a file (e.g. request.txt)
7. Make sure to install "ffuf" if it's not available and the "rockyou" wordlist (you can choose a different wordlist if it contains your password)
8. Open the terminal and type the following command `"ffuf -request request.txt -request-proto http -w [path to rockyou.txt] -mc 200 -s"` and press enter
9. Wait sometime and you should see the password in the output of the terminal (if the password in the wordlist)

- **Impact:** Lack of rate limiting in the login form can result in brute forcing passwords for any account which leads to ATO (account take over). Also it can be chained with another vulnerability (lack of rate limiting in registration form) resulting in mass ATO (account take over)
- **Remediation Advice:** One of the common protection mechanisms include (locking out a targeted account).

- **Risk Rating:** CVSS v4.0 Score: 7.2 / High
- **Summary:** Using SQLi it's possible to login with deleted account marked with the flag (deletedAt) in the database. This an access of a user's sensitive data and violation of the confidentiality.
- **Background:** Improper authentication is when an actor claims to have a given identity, the product does not prove or insufficiently proves that the claim is correct.
- **Technical Details & Evidence:**

- **Evidence**

- Improper handling of deleted accounts chained with SQLi leads to unauthorized access to deleted account



- Payload: test' or deletedAt is not null -- -
- Endpoint: POST /rest/user/login

- **Reproducing steps**

1. Open log in page
2. Use the following payload in the email field (' or deletedAt is not null -- -) and type any password
3. Click "Login" and you should log in to the first returned account

- **Impact:** The access to user's sensitive data is a violation of the confidentiality of the application and might lead to other sensitive data access after log in to the account.
- **Remediation Advice:** Usage of an authentication framework or library such as the OWASP ESAPI Authentication feature can help mitigate this vulnerability.

41. **Title:** Enumerating valid emails through registration form due to lack of rate limiting

- **Risk Rating:** CVSS v4.0 Score: 2.3 / Low
- **Summary:** Due to lack of rate limiting it's possible to use automated tools or scripts to enumerate valid emails in the registration form. On it's own it doesn't carry significant risk,

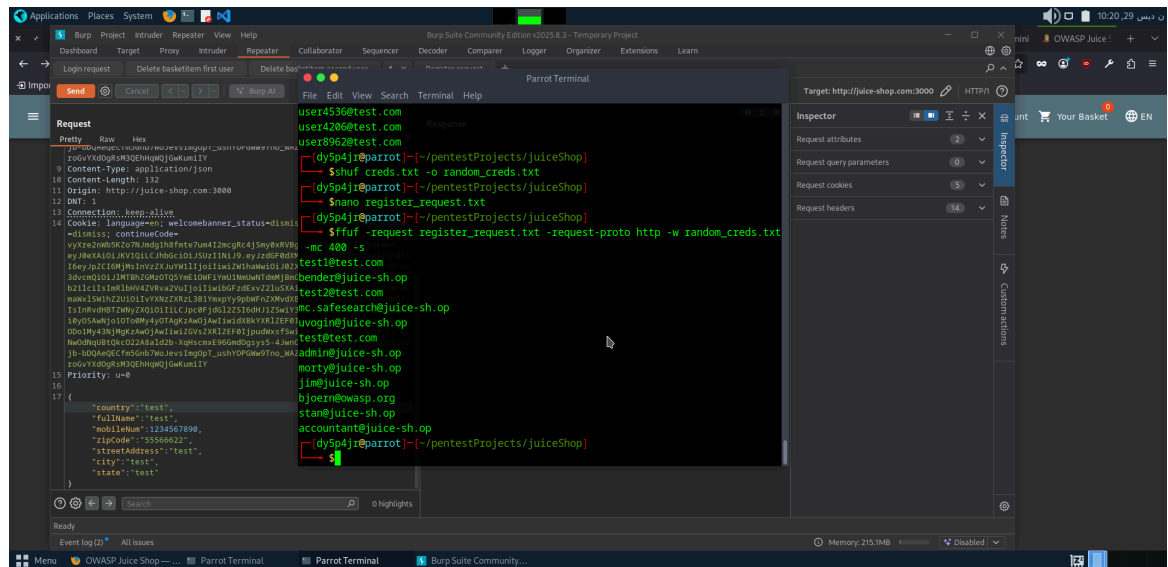
however it can be chained with other vulnerabilities (e.g. lack of rate limiting in login page) causing a mass ATO (account take over).

- **Background:** Improper restriction of excessive authentication attempts is when application does not implement sufficient measures to prevent multiple failed authentication attempts within a short time frame.

- **Technical Details & Evidence:**

- **Evidence**

- Lack of rate limiting allows of enumerating valid emails in the data base, chained together with login lack of rate limiting could leads to massive leak of the data base credentials and mass ATO



- Payload: ffuf -request register_request.txt -request-proto http -w random_creds.txt -mc 400 -s
 - Endpoint: POST /api/Users/

- **Reproducing steps**

1. Open the registration page
2. Register an account and go to the registration page again
3. Open burp suite or any proxy of your choice and make sure "intercept is on"
4. Fill in the registration form with the registered email and any data for the rest and click "Register" to intercept the request
5. Find "POST /api/Users/" endpoint and right click it and choose "send to repeater"
6. Navigate to repeater tab and change the "email" parameter's value to "FUZZ" in the JSON data body
7. Copy the entire request and paste it into a file (e.g. request.txt)
8. Make sure to install "ffuf" if it's not available
9. Use the following command in the terminal to create an email list "for i in {1..10000}; do echo "user*i*@test.com"; done > emails.txt" in linux

10. Add your registered email to the list and type the following command in the terminal to shuffle the list "shuf emails.txt -o emails_shuffled.txt" in linux
11. Open the terminal and type the following command "ffuf -request request.txt -request-proto http -w [path to emails_shuffled.txt] -mc 400 -s" and press enter
12. Wait sometime and you should see the registered email in the output of the terminal

- **Impact:** Lack of rate limiting doesn't carry significant risk on it's own, however it can be chained with other vulnerabilities (e.g. lack of rate limiting in login page) causing a mass ATO (account take over).
- **Remediation Advice:** One of the common protection mechanisms include (implementing a timeout).

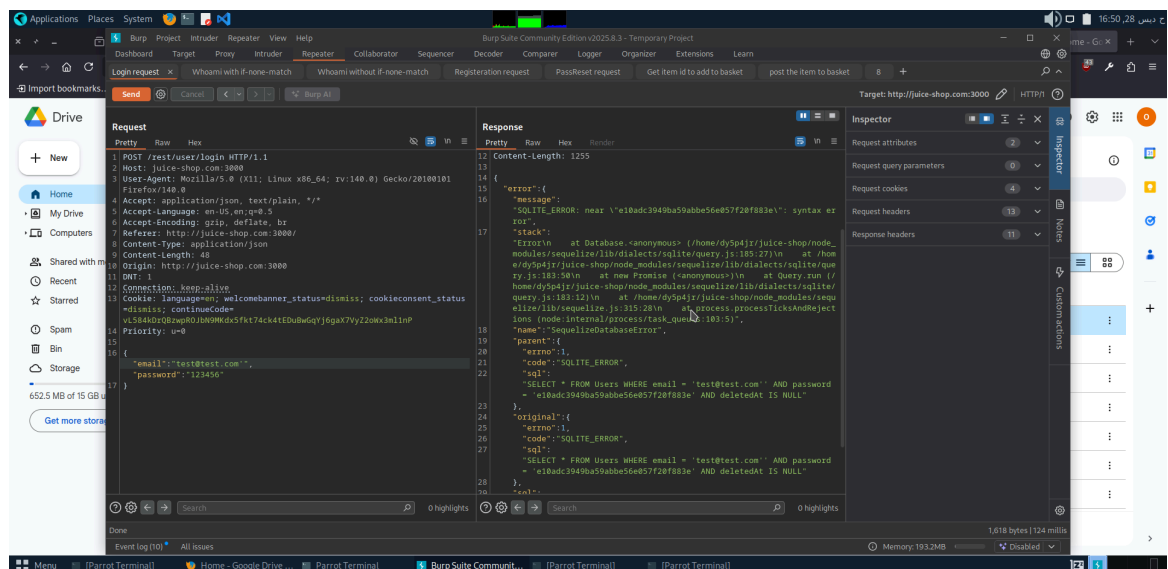
2.6 A10:2025 - Mishandling of Exceptional Conditions

42. **Title:** Verbose error message exposing database type and SQL query in the login endpoint

- **Risk Rating:** CVSS v4.0 Score: 5.3 / Medium
- **Summary:** Injecting a single quote (') in the "POST /rest/user/login" endpoint cause a verbose error message that reveals sensitive information on the database type and the SQL query. It gives a threat actor more than enough information to craft his exploit.
- **Background:** Generation of error message containing sensitive information is when application generates an error message that includes sensitive information about its environment, users, or associated data.
- **Technical Details & Evidence:**

- **Evidence**

- Verbose error message exposing database type and SQL query statement after injecting a single quote (') in the email field of the login form



- Payload: test@test.com

- Endpoint: POST /rest/user/login

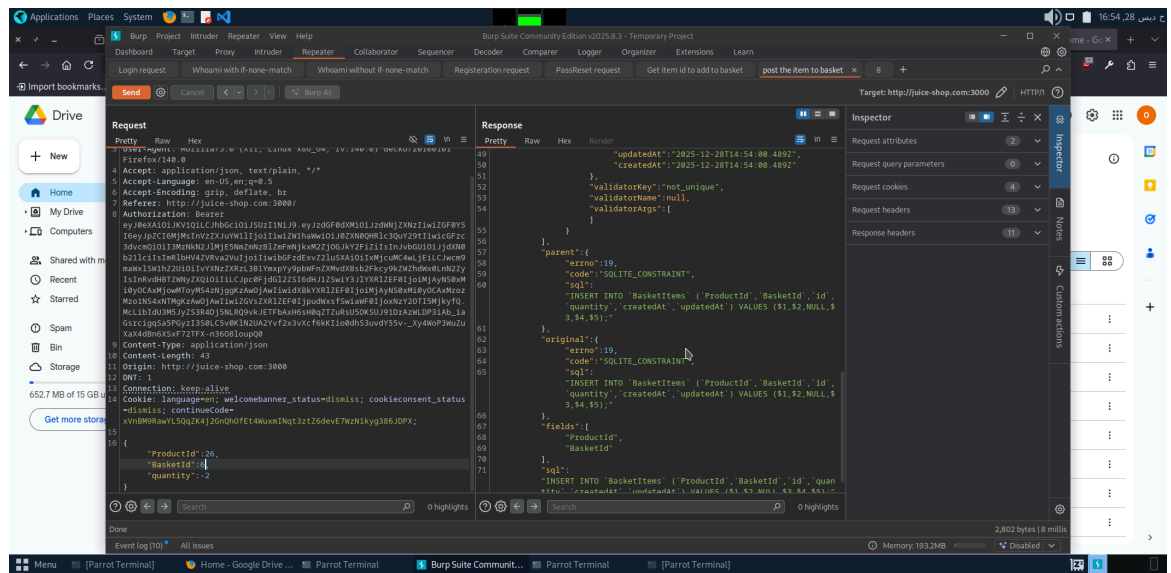
- **Reproducing steps**

1. Open the login page
2. Open burp suite or any preferred web proxy and make sure that "intercept is on"
3. Fill in the login form and click "Login" to intercept the request
4. Find the "POST /rest/user/login" endpoint and right click it then choose "send to repeater"
5. Navigate to repeater tab and inject a single quote (') in the "email" parameter in the JSON data body and click send
6. You should see the verbose error message in the response

- **Impact:** A verbose error message reveals a lot of information to any threat actor which can help crafting the exploit and fine tune it causing further damage and abuse of other vulnerabilities (e.g. SQLi).
- **Remediation Advice:** Ensure that error messages only contain minimal details that are useful to the intended audience and no one else. The messages need to strike the balance between being too cryptic (which can confuse users) or being too detailed (which may reveal more than intended).

43. **Title:** Verbose error message exposing SQL database type and the query statement in add to basket functionality

- **Risk Rating:** CVSS v4.0 Score: 5.3 / Medium
- **Summary:** Changing the "ProductId" to a wrong value in the "POST /api/BasketItems/" endpoint cause a verbose error message that reveals sensitive information on the database type and the SQL query. It gives a threat actor more than enough information to craft his exploit.
- **Background:** Generation of error message containing sensitive information is when application generates an error message that includes sensitive information about its environment, users, or associated data.
- **Technical Details & Evidence:**
 - **Evidence**
 - Verbose error message revealing the SQL database type and the query statement



- Payload: "ProductId":24
- Endpoint: POST /api/BasketItems/

• Reproducing steps

1. Log in with any account
2. Open burp suite or any preferred web proxy and make sure that "intercept is on"
3. Click "Add to basket" to intercept the request
4. Forward the first "GET /rest/basket/[id]" request then find the "POST /api/BasketItems" endpoint and right click it then choose "send to repeater"
5. Navigate to repeater tab and change the "Productid" parameter in the JSON data body to a wrong value that doesn't exist and click send
6. You should see the verbose error message in the response

- **Impact:** A verbose error message reveals a lot of information to any threat actor which can help crafting the exploit and fine tune it causing further damage and abuse of other vulnerabilities (e.g. SQLi).
- **Remediation Advice:** Ensure that error messages only contain minimal details that are useful to the intended audience and no one else. The messages need to strike the balance between being too cryptic (which can confuse users) or being too detailed (which may reveal more than intended).

44. Title: Verbose error message leaking the database type in feedback functionality

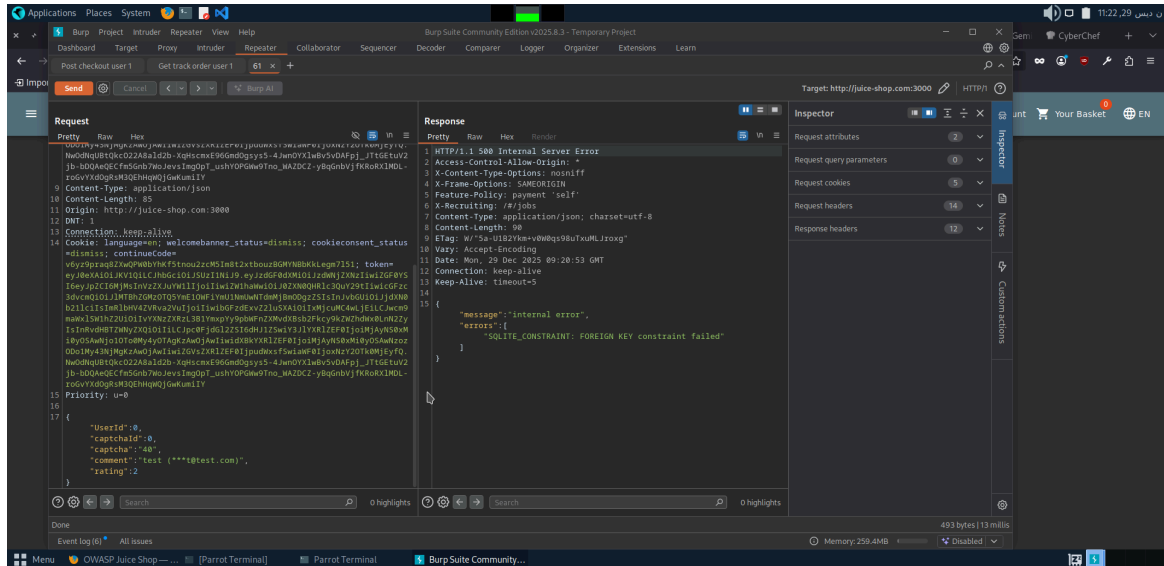
- **Risk Rating:** CVSS v4.0 Score: 5.3 / Medium
- **Summary:** Changing the "UserId" to a wrong value in the "POST /api/Feedbacks/" endpoint cause a verbose error message that reveals sensitive information on the database type. It gives a threat actor more than enough information to craft his exploit.
- **Background:** Generation of error message containing sensitive information is when application generates an error message that includes sensitive information about its

environment, users, or associated data.

- **Technical Details & Evidence:**

- **Evidence**

- Verbose error message leaking the data base technology



- Payload: "UserId":0
 - Endpoint: POST /api/Feedbacks/

- **Reproducing steps**

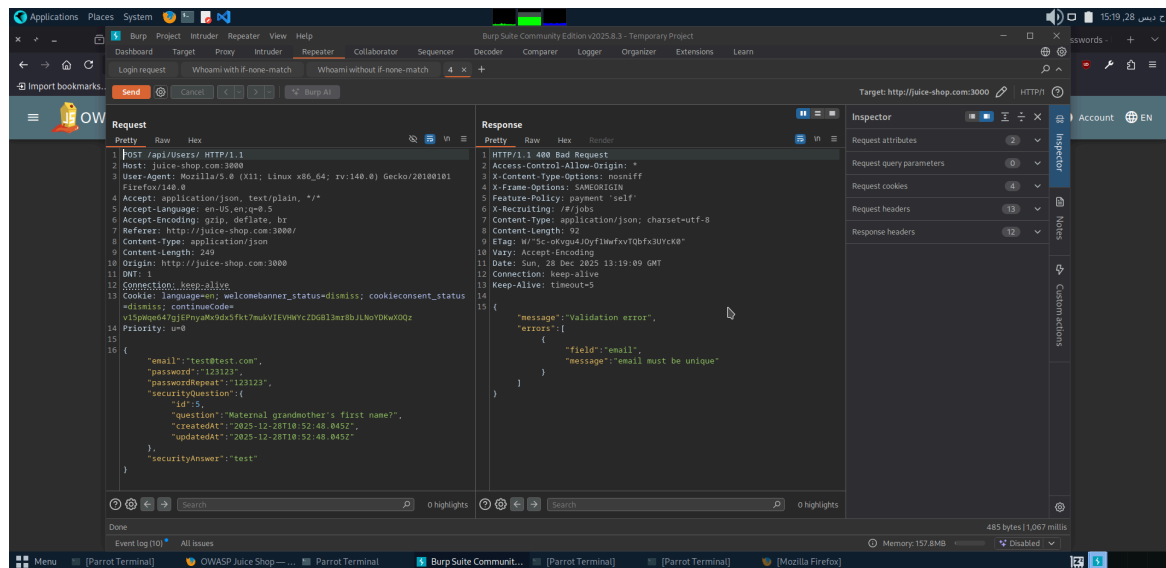
1. Log in with any account
2. Navigate to "feedbacks" page
3. Open burp suite or any preferred web proxy and make sure that "intercept is on"
4. Fill in the feedbacks form and click submit to intercept the request
5. Find the "POST /api/Feedbacks/" endpoint and right click it then choose "send to repeater"
6. Navigate to repeater tab and change the "UserId" parameter in the JSON data body to a wrong value that doesn't exist and click send
7. You should see the verbose error message in the response

- **Impact:** A verbose error message reveals a lot of information to any threat actor which can help crafting the exploit and fine tune it causing further damage and abuse of other vulnerabilities (e.g. SQLi).
- **Remediation Advice:** Ensure that error messages only contain minimal details that are useful to the intended audience and no one else. The messages need to strike the balance between being too cryptic (which can confuse users) or being too detailed (which may reveal more than intended).

45. **Title:** Verbose error message exposing valid email in register functionality

- **Risk Rating:** CVSS v4.0 Score: 2.3 / Low

- **Summary:** Verbose error message exposing valid emails when register a duplicated email. This is a low finding but can cause massive ATO (account take over) if chained with another finding like brute forcing in the login page.
- **Background:** Generation of error message containing sensitive information is when application generates an error message that includes sensitive information about its environment, users, or associated data.
- **Technical Details & Evidence:**
 - **Evidence**
 - Verbose error message exposing valid email already registered



- Payload: test@test.com
- Endpoint: POST `/api/Users/`
- **Reproducing steps**
 1. Open the registration page
 2. Register one account using non registered email address (e.g. test@test.com)
 3. Try create another account with the same email address (test@test.com)
 4. You should get an "email must be unique" message
- **Impact:** A verbose error message reveals a valid email is a low finding on it's own but can be chained with another finding like brute forcing on the login page can cause a massive ATO (account take over).
- **Remediation Advice:** Ensure that error messages only contain minimal details that are useful to the intended audience and no one else. The messages need to strike the balance between being too cryptic (which can confuse users) or being too detailed (which may reveal more than intended).

3. Appendices

3.1 Assessment Scope

- The assessment was successfully covering all of the **27 functionalities** with **total of 45 findings**, however a later reassessment might be required to cover all the attack scenarios to ensure a full comprehensive coverage of the application security.

3.2 Assessment Artefacts

1. Testing Accounts: [test@test.com, test1@test.com]
2. Files uploaded: [webshell.php, shell.php, fileSizePOC.txt]
3. several reviews through testing accounts